Quantum-inspired gravitational search algorithm-based low-price binary task offloading for multi-users in unmanned aerial vehicle-assisted edge computing systems

Santanu Ghosh
National Institute of Technology Sikkim https://orcid.org/0009-0004-4451-0878
Dr. Pratyay Kuila
pratyay_kuila@yahoo.com
National Institute of Technology Sikkim https://orcid.org/0000-0002-4549-3246
Marlom Bey
National Institute of Technology Sikkim https://orcid.org/0000-0002-3554-9381
Md Azharuddin
Aliah University, Kolkata https://orcid.org/0000-0002-1310-4541

Research Article

Keywords: Quantum agent, delay, energy, price, edge computing, binary task offloading.

Posted Date: November 7th, 2024

DOI: https://doi.org/10.21203/rs.3.rs-5403941/v1

Additional Declarations: The authors declare no competing interests.

Quantum-inspired gravitational search algorithm-based low-price binary task offloading for multi-users in unmanned aerial vehicle-assisted edge computing systems

Santanu Ghosh[a], Pratyay Kuila[a,*], Marlom Bey[a], Md Azharuddin[b]

Department of Computer Science & Engineering

[a]*National Institute of Technology Sikkim, Ravangla-737139, India*
[b]*Aliah University, Kolkata-700156, India*

Abstract

Unmanned aerial vehicles (UAVs)-assisted edge computing (EC) brings computing resources closer to smart mobile devices (SMDs). Users of SMDs with limited resources can experience the flavor of computation and data-intensive applications. Users conserve energy by offloading resource-intensive tasks to edge servers (ESs) or UAVs. While offloading may introduce communication delays, leveraging ample wireless bandwidth can mitigate this issue by facilitating faster data transmission rates. However, mobile service providers (MSPs), facilitating offloading infrastructure, impose charges on users for bandwidth usage. Efficient utilization of limited computation units (CUs) is crucial. This paper introduces a binary task offloading (BTO) model for multi-user in multi-UAV-assisted EC systems using a quantum-inspired gravitational search algorithm (QGSA). Quantum encoding and decoding of the agents are provided. The decoding phase uses a novel hashing. The fitness function considers energy, delay, price, and resource utilization. Two penalties are incorporated with the fitness to discard the decoded agent that violates the constraints of the BTO model. The proposed QGSA ensures polynomial time execution. The proposed work is extensively simulated in multiple scenarios and compared with existing approaches. It is observed that the QGSA outperformed other algorithms in terms of delay, energy, resource utilization, and price. Analyses of convergence and statistics are performed.

Keywords: Quantum agent, delay, energy, price, edge computing, binary task offloading.

1. Introduction

1.1. Background

With the evolvement of 6G and 5G networks, smart mobile devices (SMDs) increasingly require real-time computation and processing of high-volume, data-intensive tasks despite their resource constraints (Yuan et al., 2024; Zolghadri et al., 2024). Edge computing systems (ECS), aided by unmanned aerial vehicles (UAVs), offer a solution by allowing SMDs to access computational facilities and run data-intensive applications through offloading to edge servers (Ghosh and Kuila, 2023; Song et al., 2023). Mobile service providers (MSPs) play a crucial role in facilitating this

*Corresponding author

Email addresses: phcs210001@nitsikkim.ac.in (Santanu Ghosh), pratyay_kuila@nitsikkim.ac.in (Pratyay Kuila), phcs220023@nitsikkim.ac.in (Marlom Bey), md.azharuddin@aliah.ac.in (Md Azharuddin)

process by offering the necessary infrastructure for offloading and downloading application data, usually through the imposition of service charges (Liao et al., 2021).

Task offloading in a multi-UAV-assisted ECS system enables resource-constrained SMDs to transfer computation and data-intensive tasks either to remote edge servers (ESs) or UAVs equipped with computation units (CUs) (Huda and Moh, 2022). Various task offloading techniques exist (Hortelano et al., 2023), including full task offloading (FTO) (Ghosh and Kuila, 2023), binary task offloading (BTO) (Wu et al., 2020), and partial task offloading (PTO) (Dhingan et al., 2023). In FTO, all tasks are offloaded to external CUs without provision for local computation. PTO involves dividing tasks into sub-tasks, with some computed locally and the remaining offloaded to external CUs. BTO allows tasks to be executed entirely locally or offloaded to external CUs without task division. An urban scenario is depicted in Fig. 1 to illustrate the BTO model.

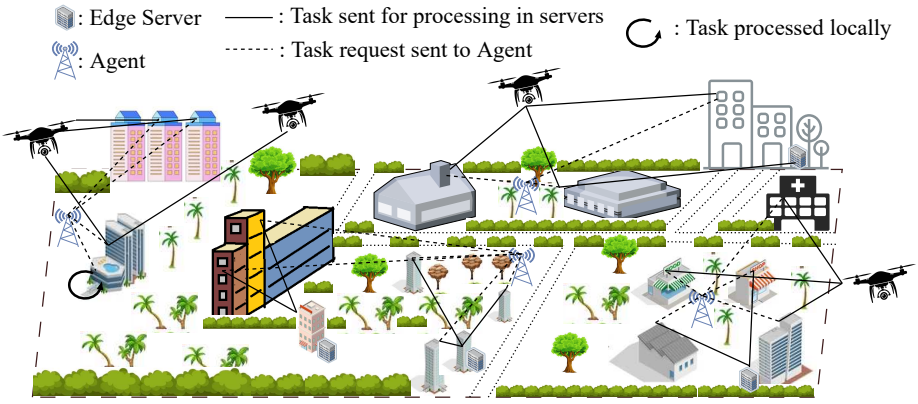


Fig. 1: Binary Task Offloading.

1.2. Motivation

While task offloading helps SMDs preserve energy, it simultaneously increases network load and introduces unavoidable latency during task transmission and reception. Although SMDs can communicate with ESs or UAVs for faster task processing, this may deplete their valuable energy due to communication distances. Additionally, such communication can increase communication delays. Hence, an intelligent decision is necessary to balance energy efficiency (Farimani et al., 2024; Nandi et al., 2024) with tolerable delay in systems (Song et al., 2022; Zhang et al., 2023).

Utilizing the ample bandwidth of the wireless channel can mitigate communication delays. However, MSPs impose charges on users for bandwidth usage and energy consumption of CUs (Liao et al., 2021). Therefore, it is also important to trade off between the price and experienced delay in the system (Qiao et al., 2022). Moreover, efficient task allocation to all CUs is crucial for optimizing computational resource utilization. Given the limited number of UAVs and ESs, tasks from SMDs must be distributed efficiently to minimize overall delay and energy consumption across UAVs, ESs, and SMDs. This research aims to contribute to binary task offloading for multiple users in multi-UAV-assisted ECS.

The challenge of BTO for multiple users in multi-UAV-assisted ECS networks falls under the dominion of non-deterministic polynomial complete (NP-complete) problems. Therefore, devising an algorithm to achieve the optimal solution within polynomial time constraints is impractical. Exploring exponential algorithms for such problems is not desirable due to their excessive computation time. However, leveraging the potential of quantum computation, quantum-inspired evolutionary algorithms (QEAs) emerge as efficient techniques for such complex NP-complete problems.

Quantum-inspired gravitational search algorithm (QGSA) (Thakur et al., 2021) is one of the well-employed QEAs that uses the principle of laws of gravity and quantum computations. With its significant exploration and exploitation capabilities, QGSA is an ideal instrument to be employed for the BTO problem.

1.3. Author's contribution

In this research, QGSA is explored for low-price and resource-efficient BTO for multi-users in multi-UAV-assisted ECS. The main contributions are as follows:

- The BTO problem (BTOP) for multi-user in multi-UAV-assisted ECS is first mathematically formulated. It is shown that the BTOP is NP-complete.
- QGSA is designed for the BTOP. The quantum-inspired encoding and decoding of the agent are provided for such scenarios. A novel hashing is used during the decoding phase.
- The fitness function is designed to evaluate the decoded agent. The fitness is comprised of delay, energy, price, and resource utilization with respect to the utilized time of the CUs. Here, transmission, computation, and receiving delay as well as energy are considered for all the devices. Moreover, the flying and hovering energy are considered for the UAVs. The pricing of the offloading is done based on the offloading energy consumption and bandwidth utilization by the users (SMDs). Moreover, two different penalties are incorporated into the fitness function to discard the decoded agents for violating the deadlines of the tasks and the threshold energy limit of the UAVs and SMDs.
- It is shown that the proposed QGSA can be executed in polynomial time.
- An extensive simulation is performed by considering multiple edge computing scenarios. The performance comparisons are done with existing works of differential evolution (DE) (Sun et al., 2021), particle swarm optimization (PSO) (You and Tang, 2021), deep reinforcement learning (DRL) (Tang and Wong, 2020) and whale optimization algorithm (WOA) (Huynh et al., 2020). Furthermore, the conventional genetic algorithm (GA) (Ram and Kuila, 2023) and conventional gravitational search algorithm (GSA) with the proposed fitness are also compared.
- The statistical and convergence analyses are also conducted.

1.4. Organization of the paper

The paper is structured as follows. Related works are presented in Sect. 2. The system model and problem formulation are demonstrated in Sect. 3. An overview of quantum computation is given in Sect. 4 followed by an overview of GSA and QGSA in Sect. 5. The proposed QGSA is provided in Sect. 6. The simulation results are discussed in Sect. 7. Sect. 8 concludes with future research.

2. Related Works

Task offloading in ECS is addressed using heuristics, meta-heuristics, reinforcement learning (RL), and mathematical optimization methods.

A heuristic task migration computation offloading (TMCO) strategy on delay and cost (handover and migration) was put out by Qiao et al. (2022) using a BTO model. Zhang et al. (2019) utilized an iterative technique to optimize bit allocation, time slot scheduling, power allocation, and

UAV trajectory design to decrease energy consumption in communication, computing, and UAV flying. Ghosh and Kuila (2023) have worked on minimizing the delay, energy, and load balance of CUs using GSA. Sun et al. (2021) have used DE to enhance the energy efficiency of an ECS. A population was initialized at random. Every population vector's fitness (energy efficiency) is computed. The agents are then subjected to mutation, crossover, and selection processes to find a more effective vector. You and Tang (2021) have used PSO for FTO. The tasks are efficiently offloaded from resource-constrained edge devices to ESs, ensuring energy efficiency and low delay. The initial position and the velocity of particles are initialized randomly. The fitness value is calculated by considering delay, energy, and penalty. Li et al. (2020) utilized the fireworks algorithm (FWA) to reduce overall delay in UAV-enabled fog computing networks for FTO. Li and Zhu (2020) successfully reduced task completion time using GA. Zhu and Wen (2021) developed a BTO technique for ECS using an improved GA (IGA) to optimize task execution delay and energy usage. Huynh et al. (2020) have used the WOA technique to minimize delay and energy in a BTO environment. In the initialization phase, random search agent positions are chosen, and resource allocations are calculated. Fitness is then calculated to determine the best value among possible solutions. The position of each agent and parameters are updated, resulting in the best possible solution. Zhang and Yan (2023) have used an improved traditional simulated annealing algorithm (ISA), considering the computation delay and energy for binary offloading the tasks.

Tang and Wong (2020) has used DRL to make BTO decisions based on delay and energy consumption. Tang and Wong (2020) determines the state-action pair of a mobile device, and each device selects an action for a new task. Each mobile device seeks to learn a mapping from every state-action pair to a Q-value. A neural network determines the mapping. To reduce the anticipated long-term cost, each device chooses the course of action that results in the lowest Q-value under its current state. Wang et al. (2021) have proposed a deep deterministic policy gradient (DDPG)-based partial computation offloading technique to reduce processing delay, offloading ratio, UAV flight angle, and flight speed for UAV-assisted mobile edge computing (MEC). Zhang et al. (2023) have proposed a PTO approach using DDPG to jointly optimize energy, time, and cost. Using the double RL computation offloading (DRLCO) technique, Liao et al. (2023) devised a BTO strategy to reduce both latency and energy usage. Bozkaya (2023) has used the Hidden Markov Model (HMM) for taking the FTO decision based on delay. Whereas, the research by (Bozkaya et al., 2023) has developed a Proof of Evaluation-based Secure Energy and Delay-Aware Task Scheduling Algorithm (PoE-SED-TSA) for taking BTO decision based on energy and delay. (Liu et al., 2023) has developed an FTO decision for a UAV-assisted MEC scenario based on delay and energy consumption. They put forth a DRL strategy based on attention mechanisms and multi-agent proximal policy optimization (MAPPO) with beta distribution and attention mechanism. A multi-cells multi-user computation offloading and multi-resources allocation (M2TORA) model based on MAPPO was suggested by (Zeng et al., 2024) to determine PTO utilising delay and energy usage.

Task offloading can use a mathematical optimization method that distributes computing jobs among dispersed resources, considering various restrictions and goals. Xu and Xu (2023) have utilized the Lagrangian dual method (LDM) to reduce energy consumption in UAV-enabled MEC systems. He et al. (2021) have employed an iterative algorithm (IA) using Lagrangian dual decomposition to optimize resource allocation in UAV-assisted vehicles. Liao et al. (2021) propose a DE technique for UE pricing, utilizing parked vehicles for multi-user edge computing, aiming to reduce energy consumption and increase income for MSPs. Additionally, a Q-learning algorithm is used to help the DE algorithm choose the appropriate unit service pricing. In the study by Fan et al. (2021), the first subproblem uses a greedy-and-search-based strategy to evaluate where to carry out the computing activities. In contrast, the second subproblem uses a golden section search to estimate the resource price.

Table 1: Overall Survey (section: 2)

Technique	Offloading Model	Delay	Energy	Price	Resource Utilization
GSA (Ghosh and Kuila, 2023)	FTO	✓	✓	✗	✗
DE (Sun et al., 2021)	FTO	✓	✓	✗	✗
PSO (You and Tang, 2021)	FTO	✓	✓	✗	✗
WOA (Huynh et al., 2020)	BTO	✓	✓	✗	✗
TMCO (Qiao et al., 2022)	BTO	✓	✗	✓	✗
Heuristic (Zhang et al., 2019)	FTO	✗	✓	✗	✗
FWA (Li et al., 2020)	FTO	✓	✗	✗	✗
GA (Li and Zhu, 2020)	PTO	✓	✗	✗	✗
IGA (Zhu and Wen, 2021)	BTO	✓	✓	✗	✗
HMM (Bozkaya, 2023)	FTO	✓	✗	✗	✗
ISA Zhang and Yan (2023)	BTO	✓	✓	✗	✗
PoE-SED-TSA (Bozkaya et al., 2023)	BTO	✓	✓	✗	✗
DRL Tang and Wong (2020)	BTO	✓	✓	✗	✗
DDPG (Wang et al., 2021)	PTO	✓	✗	✗	✗
DDPG (Zhang et al., 2023)	PTO	✓	✓	✗	✗
DRLCO (Liao et al., 2023)	FTO	✓	✓	✗	✗
LDM (Xu and Xu, 2023)	FTO	✗	✓	✗	✗
IA (He et al., 2021)	PTO	✓	✗	✗	✗
DE and Q-Learning (Liao et al., 2021)	BTO	✗	✓	✓	✗
MAPPO (Liu et al., 2023)	FTO	✗	✓	✗	✗
M2TORA (Zeng et al., 2024)	PTO	✓	✓	✗	✗
QGSA (this paper)	BTO	✓	✓	✓	✓

Despite a significant study that has been done on BTO, there is still a certain scope to be considered as highlighted in Table 1. Moreover, the proposed work has the following novelties in contrast to the existing algorithms.

- Transmission and computation delay were considered in (Ghosh and Kuila, 2023; Sun et al., 2021; You and Tang, 2021; Qiao et al., 2022; Li et al., 2020; Zhu and Wen, 2021; Wang et al., 2021; Zhang et al., 2023; Liao et al., 2023; He et al., 2021; Bozkaya, 2023; Bozkaya et al., 2023; Zeng et al., 2024). Few researches from Table 1 have considered the receiving delay. In this research, the transmission, computation, and receiving delays are considered for all the CUs.

- This research has considered propulsion, transmission, receiving, and computation energy while doing task offloading. Transmission and computational energy were considered in (Ghosh and Kuila, 2023; Sun et al., 2021; You and Tang, 2021; Zhang et al., 2019; Zhu and Wen, 2021; Zhang et al., 2023; Liao et al., 2023; Xu and Xu, 2023; Liu et al., 2023; Bozkaya et al., 2023; Zeng et al., 2024). Receiving energy was considered by (Ghosh and Kuila, 2023; Huang et al., 2019). Propulsion energy was considered by (Ghosh and Kuila, 2023; Liu et al., 2023). Propulsion energies, which include both hovering and flying energies, are crucial for efficient propulsion in UAVs.

- In most of the works (Li et al., 2020; Zhao et al., 2019; Song et al., 2020), the solution is encoded for the tasks of a single SMD only. Therefore, the system with multi-users needs to execute the algorithm individually for each user (SMD). The researches in (Sun et al., 2021; Zhu and Wen, 2021; Wang et al., 2021) have considered multiple tasks to be executed locally or offloaded in a single CU only. In contrast, the proposed algorithm uses a novel agent representation for scenarios with multi-users, multi-UAVs, and edge servers.

- Furthermore, the solution in (You and Tang, 2021; Zhao et al., 2019; Song et al., 2020) is encoded with either 0, 1, or other integer values, leading to potential information loss during the solution update phases. In contrast, the proposed QGSA utilizes quantum-encoding based on floating values to address the issue of information loss inherent with the integer-coded solution representations of (You and Tang, 2021; Zhao et al., 2019; Song et al., 2020).

3. System Models and Problem Formulation

3.1. System Model

Consider an ECS with three layers. The initial layer comprises J SMDs responsible for task generation. The second layer involves Y edge servers (ESs), while the third layer incorporates K UAVs, as illustrated in Fig. 2. It is assumed that the SMDs have sufficient computation and communication capabilities to directly communicate with the ESs and UAVs. Each SMD generates one independent and atomic service (task) request. These requests can be locally executed on the respective SMDs or fully offloaded to any one of the ESs or UAVs. Therefore, a task can be executed by any one of the possible $(K + Y + 1)$ units, known as CUs. Tasks are quota/threshold oriented, requiring completion within a fixed time, with quotas set at different levels for users. It is important to note that the UAVs are operated by limited battery power. Therefore, all responsibilities assigned to the UAVs must be accomplished within the constraints of their limited battery capacity.

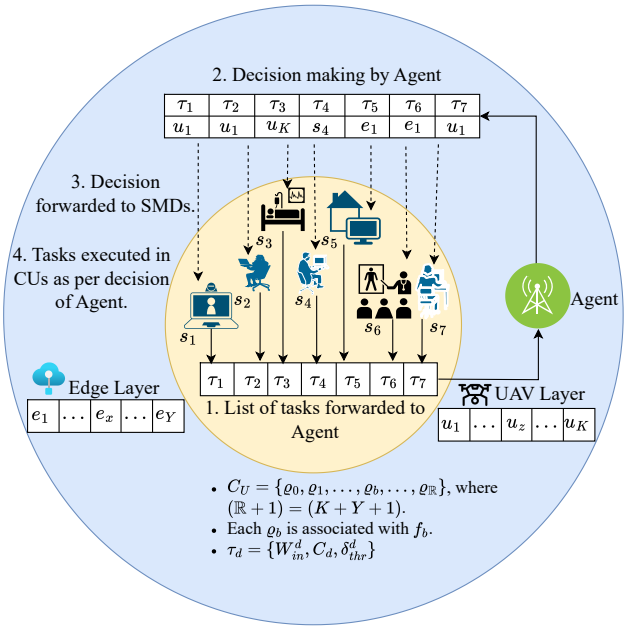


Fig. 2: System Model.

An intelligent agent (IA) at the SMD layer is assumed to collect task requests from the SMDs. The IA subsequently evaluates these task requests to determine whether a task should be offloaded or not. If so, the IA selects the appropriate ES or UAV using the proposed algorithm. It is assumed that the IA has all the information about the CUs that are available for computation. The subsequent models and the offloading problem are described using the following terminologies and notations:

- $\mathbb{S} = \{s_1, s_2, ..., s_d, ..., s_J\}$ denotes the set of J number of SMDs.
- $\mathbb{U} = \{u_1, u_2, \ldots, u_z, \ldots u_K\}$ denotes the set of K number of UAVs.
- $E = \{e_1, e_2, \ldots, e_x, \ldots e_Y\}$ denotes the set of Y number of ESs.
- $C_U = \{\varrho_0, \varrho_1, \ldots, \varrho_b, \ldots, \varrho_{\mathbb{R}}\}$ represents the set of $(\mathbb{R}+1) = (K+Y+1)$ CUs for the execution of a task. This set comprises K UAVs, Y ESs, and the originator of the task itself. The term ϱ_0 corresponds to the SMD generating the task. The set $\{\varrho_1, \ldots, \varrho_K\}$ pertains to the UAVs, and $\{\varrho_{K+1}, \ldots, \varrho_{\mathbb{R}}\}$ pertains to the ESs.
- $T = \{\tau_1, \tau_2, ..., \tau_d, ..., \tau_J\}$ denotes the set of J number of tasks that are generated by the J SMDs. The τ_d is assumed to be generated from the SMD s_d.
- Each task τ_d can be represented using a three-tuple notation as $\tau_d = \{W_{in}^d, C_d, \delta_{thr}^d\}$ (Islam et al., 2021). The data size is represented by W_{in}^d. The number of CPU cycles per bit required to process one bit is denoted by C_d. On the other hand, δ_{thr}^d denotes the hard-deadline time/threshold for τ^d to be completed.
- Let ζ_{db} be a Boolean variable to address the offloading decision of τ_d as follow:

$$\zeta_{db} = \begin{cases} 1, & \text{if } \tau_d \text{ is offloaded to the CU } \varrho_b \\ 0, & \text{Otherwise.} \end{cases} \tag{1}$$

  Therefore, $\zeta_{d0} = 1$ indicates that τ_d is executed at the CU ϱ_0, i.e., the SMD s_d itself.
- The SMDs and UAVs have limited energy as they run on batteries. The maximum energy consumed by the SMD (s_d) during the offloading must not exceed the available energy (E_{thr}^d) in the SMD. The energy used by the UAVs needs to stay under the threshold ($E_{uav-thr}$).

Table 2: Notation and meanings (section: 3).

Notation	Meaning	Notation	Meaning
τ_d	d^{th} task	f_b	CPU frequency of a CU
δ_{total}	Total delay	J	Number of task
ϱ_b	b^{th} CU	P_{total}	Total price
E_{SMD}^d	Energy of s_d	$\mathbb{R}$	Total CUs
E_{off}	Offloading energy of u_z	s_d	d^{th} SMD
E_{ES}^b, E_{UAV}^b	Energy of a ES and a UAV	u_z	z^{th} UAV
C_d	Computation capacity required for a task	E_{total}	Total energy

3.2. Delay Model

Delay minimization is most required while offloading tasks for critical situations (Yu et al., 2020). For a critical zone scenario, different types of delays are considered as discussed below.

- *Transmission delay:* The delay to transmit the task τ_d from the SMD s_d to a CU ϱ_b is represented as $\delta_t^{db} = \frac{W_{in}^d}{R^{db}}$, where W_{in}^d denotes the input data volume of τ_d in bits, and $R^{db} = B_d[\log_2(1 + \frac{P^d h^{db}}{\sigma^2})]$ is the data transmission rate between the wireless channel of the SMD s_d to the CU ϱ_b. B_d refers to the bandwidth of the channel selected by s_d. Let the bandwidth selected by SMD users be represented as $\{B_1, B_2, \ldots, B_d, \ldots, B_J\}$. The R^{db} is calculated as shown by Zhang et al. (2019).

- *Receiving delay:* The receiving delay (Ghosh and Kuila, 2023) to receive W_{out}^d bits of data by a CU ϱ_b can be computed as $\delta_r^{bd} = \frac{W_{out}^d}{R^{db}}$.
- *Computation delay:* The computation delay (Li and Zhu, 2020) while computing the input request in the ϱ_b is $\delta_c^{bd} = \frac{W_{in}^d \times C_d}{f_b}$, where the number of CPU cycles (CPU cycles/bit) needed to process one bit in ϱ_b is C_d. The computing capability of a ϱ_b or the number of CPU cycles per second is denoted by f_b.

Definition 3.1 (**Task completion time (δ_{Total}^d)**). The total time required to complete the task τ_d by ϱ_b is:

$$\delta_{Total}^d = \begin{cases} \delta_t^{db} + \delta_r^{bd} + \delta_c^{bd}, & \text{if } \zeta_{db} = 1, \text{ i.e., } \tau_d \text{ is offloaded to } \varrho_b. \\ \delta_c^{0d}, & \text{if } \zeta_{d0} = 1, \text{ i.e., } \tau_d \text{ is executed locally at } s_d. \end{cases} \tag{2}$$

When τ_d is computed locally (i.e., $\zeta_{d0} = 1$) at the corresponding SMD s_d, then $\delta_t^{d0} = \delta_r^{0d} = 0$. Note that δ_{Total}^d should not violate the hard-deadline δ_{thr}^d.

Definition 3.2 (**Used time of CU ($T\delta_{CU}^b$)**). The total used time of ϱ_b to complete all the offloaded tasks can be expressed as:

$$T\delta_{CU}^b = \sum_{d=1}^{J} \{\zeta_{db}.\delta_{Total}^d\} \tag{3}$$

If the CU is an SMD s_d, then the used time is when the corresponding task τ_d is locally executed, i.e., $\zeta_{d0} = 1$. In such cases, $T\delta_{SMD}^d = \zeta_{d0}.\delta_{Total}^d = \delta_c^{0d}$.

Definition 3.3 (**Maximum delay (δ_{total})**). It is the maximum of the used time of all the CUs and can be expressed as:

$$\delta_{total} = \max\{T\delta_{SMD}^d, T\delta_{CU}^b | \forall b, 1 \leq b \leq \mathbb{R}, \forall d, 1 \leq d \leq J\} \tag{4}$$

3.3. Resource Utilization Model

Here, the resource indicates the CUs as in (Ghosh and Kuila, 2023; Zhang et al., 2023). Since resources are available to execute the tasks, greater resource utilization will result in less delay. Here, the utilization ratio (U_R) of a CU is the ratio between the actual utilized time and the total delay of the system (δ_{total}).

The U_R of a CU ϱ_b can be calculated as: $U_R(CU) = \frac{T\delta_{CU}^b}{\delta_{total}}$. Here, we have a total J number of SMDs and $\mathbb{R} = (K + Y)$ number of UAVs and ESs. The total utilization for the SMDs is $\sum_{d=1}^{J} \frac{T\delta_{SMD}^d}{\delta_{total}}$ and $\mathbb{R}$ number of other CUs is $\sum_{b=1}^{\mathbb{R}} \frac{T\delta_{CU}^b}{\delta_{total}}$. Therefore, the average utilization ratio for the system with J number of SMDs and $\mathbb{R}$ number of UAVs and ESs can be formulated as:

$$U_R(Total) = \frac{1}{J + \mathbb{R}} \Big\{ \sum_{d=1}^{J} \frac{T\delta_{SMD}^d}{\delta_{total}} + \sum_{b=1}^{\mathbb{R}} \frac{T\delta_{CU}^b}{\delta_{total}} \Big\} \tag{5}$$

3.4. Energy Model

The energy is consumed due to i) transmission (while the entire task is offloaded from SMD to CU), ii) receiving (when the output of the processed task is received from the CU to the SMD), iii) computation (to compute the task), and iv) propulsion of the UAVs (for the flying and hovering of the UAVs).

- *Transmission Energy:* When a task τ_d is offloaded from the corresponding SMD s_d to a CU ϱ_b, the energy (E_t^{db}) is consumed by the SMD s_d to transmit as $E_t^{db} = P_t^d \times \delta_t^{db}$, where P_t^d is the power required by s_d to transmit a task (Zhang et al., 2019). Consequently, the output of the processed task is relayed from ϱ_b and received by s_d. The transmission energy (E_t^{bd}) consumed by ϱ_b is calculated as $E_t^{bd} = P_t^b \times \delta_r^{bd}$ (Ma and Wang, 2023).

- *Computation Energy:* It is the energy required to process the task τ_d by ϱ_b. It can be estimated as $E_c^{bd} = k \times f_b^3 \times \delta_c^{bd}$, where k is a constant (Zhang et al., 2018). When τ_d is computed locally (i.e., $\zeta_{d0} = 1$) at the corresponding SMD s_d, then it can be estimated as $E_c^{0d} = k \times f_d^3 \times T\delta_{SMD}^d$.

- *Receiving Energy:* The CU ϱ_b consumes energy (E_r^{bd}) to receive the offloaded task from the SMD s_d as $E_r^{bd} = P_r^d \times \delta_r^{bd}$. Consequently, the s_d consumes energy (E_r^{db}) to receive the output of the task, sent by the ϱ_b as $E_r^{db} = P_r^b \times \delta_t^{db}$. Here, P_r^d and P_r^b are the power required to receive by the s_d and ϱ_b respectively.

Definition 3.4 (**Total offloading energy consumption (E_{off})**). The total energy consumption due to task offloading by all the devices can be formulated as:

$$E_{off} = \sum_{d=1}^{J} \Big\{ \zeta_{d0}.E_c^{0d} + \sum_{b=1}^{\mathbb{R}} \{ \zeta_{db}(E_t^{db} + E_r^{db} + E_r^{bd} + E_t^{bd} + E_c^{bd}) \} \Big\} \tag{6}$$

The first part (i.e., $\zeta_{d0}.E_c^{0d}$) of the eq. (6) indicates the local execution of the tasks τ_d at SMD s_d, and in such cases, $E_t^{db} = E_r^{bd} = 0$. The next part is the energy consumption while offloading at ESs and UAVs.

Definition 3.5 (**Energy consumption by a SMD (E_{SMD}^d)**). The total energy consumption by the SMD s_d is given below:

$$E_{SMD}^d = \begin{cases} E_t^{db} + E_r^{bd}, & \text{if } \zeta_{db} = 1, \text{ i.e., } s_d \text{ offloads the task to } \varrho_b. \\ E_c^{0d}, & \text{if } \zeta_{d0} = 1, \text{ i.e., } s_d \text{ executes locally.} \end{cases} \tag{7}$$

Definition 3.6 (**Energy consumption by an ES (E_{ES}^b)**). The total energy consumed by an ES is the offloading energy consumption for the tasks that are offloaded to the ES. It is estimated as:

$$E_{ES}^b = \Big\{ \sum_{d=1}^{J} \zeta_{db}(E_r^{db} + E_t^{bd} + E_c^{bd}) \Big\}; \forall b, K+1 \leq b \leq \mathbb{R} \tag{8}$$

- *Propulsion Energy of UAVs:* Propulsion energy comprises hovering and flying energy. The hovering energy of a UAV is computed as per eq. (9).

$$E_{hov}^b = (P_0 + P_1) \times \delta_{hov}^b; \forall b, 1 \leq b \leq K \tag{9}$$

  where P_0 and P_1 represent the energy used by the rotary blade's rotation and induction per unit of time. δ_{hov}^b is hovering time of an UAV. A UAV u_z will hover until all tasks assigned to the u_z are executed. So, δ_{hov}^b is taken as $T\delta_{CU}^b$, as per eq. (3).

  The flying energy is controlled by the induced, blade profile, and parasite phase (Zhang et al., 2019; Zeng et al., 2019). The induced part is in connection with the induction power. While the blade profile part is associated with the power which rotates the rotary blades. Rotor disc

area ($\mathbb{G}$), fuselage drag ratio (d_r), solidity of the rotor (S), and air density (ρ) are associated with the parasite component. Vertical movement of UAV is reflected in the end which is related to power per unit time, P_2. The propulsion energy of a UAV is represented as:

$$E^b_{fly} = \sum_{b=1}^{K} \Big\{ \delta^b_{fly} \big[P_0 (1 + \frac{3 \parallel V_{U,xy} \parallel^2}{V^2_{tip}}) + \frac{1}{2} d_r S \rho \mathbb{G} \parallel V_{U,xy} \parallel^3 + P_1 \sqrt{(\sqrt{1 + \frac{\parallel V_{U,xy} \parallel^4}{4V_0^2}} - \frac{\parallel V_{U,xy} \parallel^2}{2V_0^2})} + P_2 \parallel V_{U,z} \parallel \big] \Big\} \tag{10}$$

where, V_0, V_{tip} represents the mean rotor-induced velocity and rotor blade tip speed during hover. $V_{U,z}, V_{U,xy}$ are vertical and horizontal velocities of the UAV. δ^b_{fly} specifies the flying time for the UAV. The flying delay (Li et al., 2021) talks about the flying time of UAV as shown in eq. (11).

$$\text{Flying Time } (\delta^b_{fly}) = \frac{D_{p1,p2}}{V_{U,xy}} \tag{11}$$

where, $D_{p1,p2}$ refers to the Euclidean distance between the hovering point $p1$ and hovering point $p2$ and $V_{U,xy}$ is the speed of UAV. So, the propulsion energy of the UAV is formulated as depicted in eq. (12).

$$E^b_{prop} = E^b_{fly} + E^b_{hov}; \forall b, 1 \leq b \leq K \tag{12}$$

Definition 3.7 (**Energy consumption by a UAV** (E^b_{UAV}))**.** The total energy consumed by a UAV is the summation of propulsion and offloading energy consumption for the tasks that are offloaded to the UAV. The total energy consumed by the UAV is:

$$E^b_{UAV} = \Big\{ E^b_{prop} + \sum_{d=1}^{J} \zeta_{db} (E^{db}_r + E^{bd}_t + E^{bd}_c) \Big\}; \forall b, 1 \leq b \leq K \tag{13}$$

Definition 3.8 (**Total Energy consumption** (E_{total}))**.** The total energy consumption in all the participating devices in task offloading can be estimated as:

$$E_{total} = E_{off} + \sum_{b=1}^{K} E^b_{prop} \tag{14}$$

3.5. Price Model

In this developing market, dynamic pricing policy is a commercial tactic to give SMDs access to scalable and economical computing resources when they need them (Han et al., 2018; Vardakas et al., 2014). According to Qiu et al. (2019), μ_i is defined by the mobile network operator (MNO) as the cost of each bandwidth unit given to the UAV operator i. Taking this idea, in this research the MSP provides $\mathbb{P}$ as the cost charged per megahertz of bandwidth. The SMD user chooses the bandwidth and pays the fee according to that choice. Liao et al. (2021) and Gupta et al. (2022) build the pricing model based on the energy consumption of servers. This research proposes a system where the pricing model is built on the amount of energy required to transfer the raw data, the data executed, and the result received back in the SMDs. The total price incurred (P_{total}) is equal to the sum of prices incurred from UAV and ES servers, as shown in eq. (15).

$$P_{total} = \underbrace{\sum_{d=1}^{J}\{\mathbb{P} \times B_d\}}_{\text{Pricing for bandwidth}} + \underbrace{\{w_1 \times E_{ES}\}}_{\text{Computation price of ES}} + \underbrace{\{w_2 \times E'_{UAV}\}}_{\text{Computation price of UAV}} \quad (15)$$

where w_1 and w_2 are the fixed unit service prices associated with the energy consumption. Here, $E_{ES} = \sum_{b=K+1}^{\mathbb{R}} E_{ES}^{b} = \sum_{b=K+1}^{\mathbb{R}} \sum_{d=1}^{J}\{\zeta_{db}(E_r^{db} + E_t^{bd} + E_c^{bd})\}$ and $E'_{UAV} = \sum_{b=1}^{K} \sum_{d=1}^{J}\{\zeta_{db}(E_r^{db} + E_t^{bd} + E_c^{bd})\}$.

3.6. Problem Formulation

The problem of low-price and resource-efficient BTO for multi-user in multi-UAV-assisted ECS (BTOP) can be stated as: "Given an ECS system with J number of SMDs with a task, therefore total J number of tasks $T = \{\tau_1, \tau_2, \ldots, \tau_d, \ldots \tau_J\}$. There are $\mathbb{R} = (Y + K)$ number of computing units (CUs) including Y ESs and K UAVs. Along with these CUs, SMDs are also available for the computation of their own task. The challenge is to delegate every task τ_d to any one of the $\mathbb{R}$ number CUs or the respective SMD s_d by reducing the energy consumption, delay in computation, idleness of CUs, and price incurred by the SMD users." The research works on the four objective functions. The objectives are:

$$\text{Objective} \quad 1: \quad \text{Minimize total delay } (\delta_{total}) \quad (16)$$

$$\text{Objective} \quad 2: \quad \text{Minimize total energy } (E_{total}) \quad (17)$$

$$\text{Objective} \quad 3: \quad \text{Minimize total price } (P_{total}) \quad (18)$$

$$\text{Objective} \quad 4: \quad \text{Maximize resource utilization of CUs } (U_R(Total)) \quad (19)$$

Subject to:

$$\text{Constraint} \quad 1: \quad \sum_{b=0}^{\mathbb{R}} \zeta_{db} = 1, \forall d, 1 \leq d \leq J \quad (20)$$

$$\text{Constraint} \quad 2: \quad \zeta_{db} \in \{0, 1\}, \forall d, 1 \leq d \leq J, \forall b, 0 \leq b \leq \mathbb{R} \quad (21)$$

$$\text{Constraint} \quad 3: \quad \mathbb{R} = (Y + K) \quad (22)$$

$$\text{Constraint} \quad 4: \quad E_{UAV}^{b} < E_{uav-thr}, \forall b, 1 \leq b \leq K \quad (23)$$

$$\text{Constraint} \quad 5: \quad \sum_{b=0}^{\mathbb{R}} \zeta_{db}.\delta_{Total}^{bd} < \delta_{thr}^{d}, \forall d, 1 \leq d \leq J \quad (24)$$

$$\text{Constraint} \quad 6: \quad E_{SMD}^{d} < E_{thr}^{d}, \forall d, 1 \leq d \leq J \quad (25)$$

The equations (16)-(19) talk about the objectives of the assumed offloading problem (PEDRP-BOP). Equations (20)-(21) ensure all the tasks are offloaded either to the CUs or to their respective s_d. As previously mentioned, the eq. (22) tells that total $\mathbb{R}$ CUs are present where there are Y number of ESs and K number of UAVs. Equations (23)-(25) states the different thresholds applied on δ_{total}, E_{UAV}, and E_{SMD}^{d} for selecting the correct solution.

Remark 3.1. Notably, the previously given issue is multi-objective and a 0, 1 integer linear programming (0,1 - ILP) problem. The offloading decision variable, ζ_{db} is restricted to be integer (0 or 1), and the objective function is linear.

Theorem 1. *The BTOP is NP-Complete.*

Proof. The 0-1 Knapsack problem (KP) is a well-known NP-complete problem. The BTOP is a generalization of the KP (KP $\leq_p$ BTOP). Here, $\leq_p$ signifies a polynomial-time reduction. It can be said that all the problems in NP are polynomial-time reducible to BTOP, i.e., $\forall A \in$ NP, $A \leq_p$ BTOP. Additionally, BTOP $\in$ NP since it can be verified in polynomial time. Thus, it can be said that BTOP is NP-complete. □

4. An Overview of Quantum Computation

Quantum computation is a field of study that is based on the principles of quantum mechanics. Some key principles and features of quantum computation include superposition, entanglement, quantum gates, etc. Traditional computers use bits to represent information, which can exist in one of two states: 0 or 1. Quantum computers, on the other hand, use quantum bits, or qubits (Han and Kim, 2002; Saad et al., 2021; Bey et al., 2024b; Zhang, 2011). Qubits can exist in multiple states simultaneously due to a phenomenon called superposition. This property enables quantum computers to perform certain types of calculations much more efficiently than classical computers.

Definition 4.1 (**Qubit**)**.** A Qubit or Q-bit is the smallest unit of information that can be in any one of the three states $|0\rangle$ or $|1\rangle$ or in a linear superposition of both.

The linear superposition of a qubit is expressed as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where, α and β are the complex numbers (Bandyopadhyay et al., 2024; Bey et al., 2024a). A qubit can be represented as: $q = \{\alpha, \beta\}$ and can be measured as 0 with probability $|\alpha|^2$ or as 1 with probability $|\beta|^2$ such that:

$$|\alpha|^2 + |\beta|^2 = 1 \tag{26}$$

Definition 4.2 (**Multi-Qubit**)**.** A multi-qubit with D number of qubits is represented as:

$$Q = [q_1, q_2, \ldots, q_D] = \left[\begin{array}{c|c|c|c|c|c} \alpha_1 & \alpha_2 & \ldots & \alpha_m & \ldots & \alpha_D \\ \beta_1 & \beta_2 & \ldots & \beta_m & \ldots & \beta_D \end{array}\right] \tag{27}$$

where, $\alpha_m^2 + \beta_m^2 = 1, \forall m = \{1, 2, \ldots, D\}$. Such D length multi-qubit represents 2^D number of states simultaneously with their respective probability. Thereby, it increases the number of possibilities.

Illustration 4.1. Considering a four-qubit system as $Q = \left[\begin{array}{c|c|c|c} \frac{1}{2} & \frac{-1}{\sqrt{2}} & \frac{\sqrt{3}}{2} & \frac{-\sqrt{3}}{2} \\ \frac{\sqrt{3}}{2} & \frac{1}{\sqrt{2}} & \frac{1}{2} & \frac{1}{2} \end{array}\right]$. There are $2^4 = 16$ number of states as: $|0000\rangle, |0001\rangle, |0010\rangle, ..., |1111\rangle$. The probability of the state $|1010\rangle$ is calculated as $(\frac{\sqrt{3}}{2})^2 \times (-\frac{1}{\sqrt{2}})^2 \times (\frac{1}{2})^2 \times (-\frac{\sqrt{3}}{2})^2 = \frac{9}{128}$.

The quantum information is typically updated using the quantum gates or Q-gates. A Q-gate is a gate that accepts a Qubit (α, β) as input and outputs a modified Qubit (α', β') with updated amplitudes. It must also comply with the normalization rule as per eq. (26).

5. An overview of GSA and QGSA

5.1. Gravitational search algorithm (GSA)

The GSA (Ghosh and Kuila, 2023) is a widely used meta-heuristic optimization technique rooted in Newton's law of gravitation. According to this law, every object in the universe exerts a gravitational force on every other object, a force that is directly proportional to their masses and inversely proportional to the distance between them. The GSA begins by initializing agents,

where a collection of these agents constitutes a population. Each agent is represented as $\vec{A}_i = \{a_{i,1}, a_{i,2}, ..., a_{i,d}, ..., a_{i,D}\}$, with D indicating the dimension and $a_{i,d}$ the d^{th} component of $\vec{A}_i$. The fitness of each agent is then evaluated to determine the quality of the agent. Based on this fitness, the mass ($M_i(t)$) of an agent $\vec{A}_i(t)$ is calculated according to eq. (28).

$$M_i(t) = \begin{cases} 1, & \text{if } (best == worst) \\ \dfrac{fitness_i(t) - worst}{best - worst}, & \text{Otherwise} \end{cases} \tag{28}$$

where, $best = min\{fitness_i(x) | \forall i, 1 \leq i \leq N_P, \forall x, 1 \leq x \leq t\}$ (for minimization problem) and $worst = max\{fitness_i(t) | \forall i, 1 \leq i \leq N_P\}$. According to Newton's law of gravitation, each agent $\vec{A}_i(t)$ applies an attractive gravitational force on every other agent $\vec{A}_j(t)$ within the population. Let $f_{i,j,d}(t)$ represents the gravitational force between $\vec{A}_i(t)$ and $\vec{A}_j(t)$ towards the d^{th} component of both the agents as follows:

$$f_{i,j,d}(t+1) = G(t) \times \frac{M_i(t) \times M_j(t)}{R_{i,j} + \measuredangle} \times (a_{j,d}(t) - a_{i,d}(t)) \tag{29}$$

where, $R_{i,j} = ||\vec{A}_i(t), \vec{A}_j(t)||$ is the Euclidean distance between $\vec{A}_i(t)$ and $\vec{A}_j(t)$. $\measuredangle$ denotes a tiny positive constant to prevent the denominator from becoming zero especially when $R_{i,j} = 0$. The $G(t)$ is the gravitational constant and can be calculated as $G(t) = G(0) \times e^{(\frac{-\mathbb{k} \times t}{MAX})}$, where $\mathbb{k}$ is a descending coefficient, MAX is the total iteration, and $G(0)$ is the starting value. The total force on the agent $\vec{A}_i(t)$ towards the d^{th} component is calculated as:

$$F_{i,d}(t+1) = \sum_{j=1, j \neq i}^{N_P} rand_j \times f_{i,j,d}(t+1) \tag{30}$$

where $rand_j$ is in the range of 0 - 1. Let us assume $\vec{\mathbf{F}}_i(t+1) = \{F_{i,1}(t+1), F_{i,2}(t+1), ..., F_{i,D}(t+1)\}$ be the total force on $\vec{A}_i(t+1)$. The acceleration of d^{th} component of $\vec{A}_i(t)$ can be calculated as:

$$ac_{i,d}(t+1) = \frac{F_{i,d}(t+1)}{M_i(t)} \tag{31}$$

The velocity is refined following the acceleration:

$$v_{i,d}(t+1) = v_{i,d}(t) + ac_{i,d}(t+1) \tag{32}$$

The position of the agent is updated following the velocity update.

$$a_{i,d}(t+1) = a_{i,d}(t) + v_{i,d}(t+1) \tag{33}$$

The process is iterated till the termination condition is met.

5.2. Quantum-inspired GSA

Quantum-inspired GSA (QGSA) leverages quantum mechanics principles, such as superposition and parallelism, to improve GSA's exploration and exploitation capabilities (Thakur et al., 2018). In QGSA, agents are represented as multi-qubit systems, with qubit angular velocity updated via rotation gates. Fig. 3 provides a schematic representation of the QGSA process. A detailed discussion of the complete QGSA workflow is presented in the following section through the proposed

work.

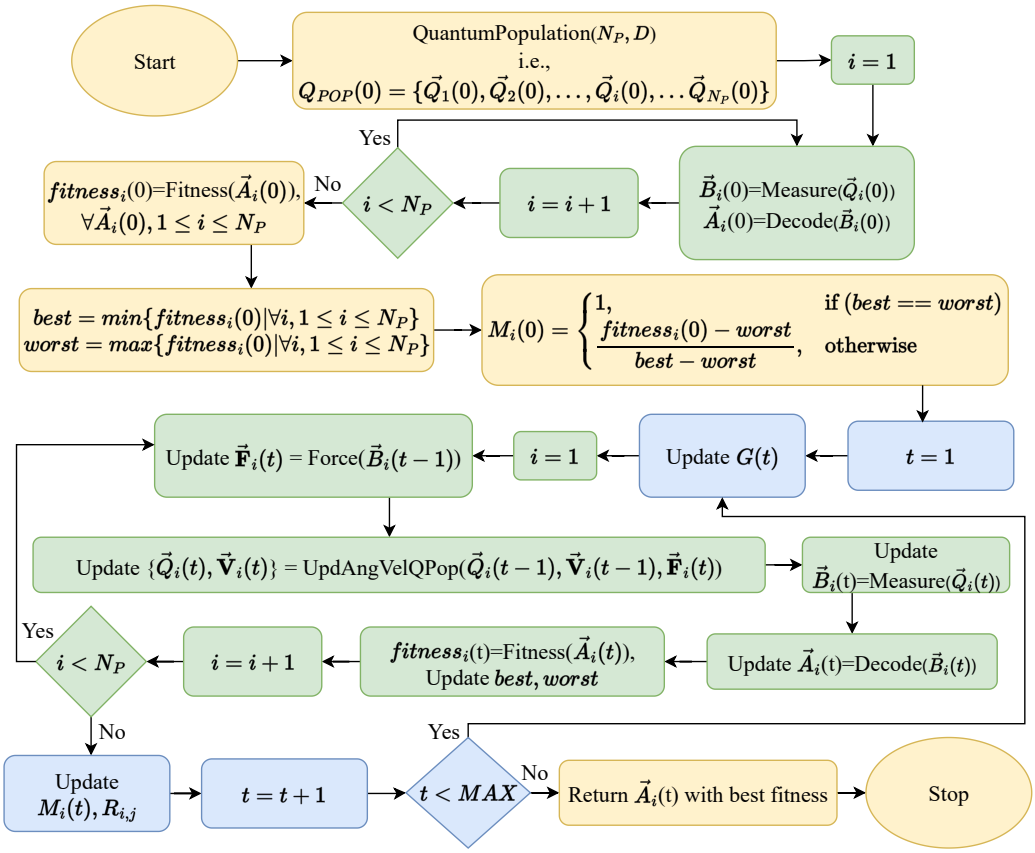


Fig. 3: QGSA flowchart.

6. Proposed QGSA for BTOP

This section provides a brief explanation of the procedures that QGSA needs to take to solve the BTOP.

6.1. Initialization of Quantum Population

The quantum population is the set of N_P number of quantum agents (QAs) and can be denoted as $\vec{Q}_{POP}(t) = \{\vec{Q}_1(t), \vec{Q}_2(t), \ldots, \vec{Q}_i(t), \ldots \vec{Q}_{N_P}(t)\}$. Here, N_P denotes the population size, and t represents the generation (initially, $t = 0$). A QA ($\vec{Q}_i(t)$) is represented as per eq. (34).

$$\vec{Q}_i(t) = [q_1, q_2, \ldots, q_m, \ldots, q_D] = \left[\begin{array}{c|c|c|c|c} \alpha_{i,1}(t) & \ldots & \alpha_{i,m}(t) & \ldots & \alpha_{i,D}(t) \\ \beta_{i,1}(t) & \ldots & \beta_{i,m}(t) & \ldots & \beta_{i,D}(t) \end{array}\right] \tag{34}$$

where D is the dimension of QAs. The QA is represented in such a way that it can ensure a complete solution to the considered BTO problem, i.e., each task is assigned to a CU. As there are ($\mathbb{R}$+1) number of CUs and J number of tasks, the dimension of QA is finalized as $D = J \times Bits$. The minimum number of bits required to represent $\mathbb{R}$ is $Bits = \lceil \log_2(\mathbb{R} + 1) \rceil$. The quantum population is produced as per the Algorithm 1 (Bey et al., 2024a).

Algorithm 1 QuantumPopulation(N_P, D)

Input: Dimension of the QA: D and Population size: N_P.
Output: Initial quantum population, $\vec{Q}_{POP}(t)$.

1: Initialize $\vec{Q}_{POP}(t)$ ={}
2: Initialize $\vec{Q}_i(t) = \{\}, \forall i, 1 \leq i \leq N_P$.
3: **for** $i = 1$ to N_P
4: **for** $m = 1$ to D
5: $x = rand[-\infty, \infty]$
6: $y = rand[-\infty, \infty]$
7: $\alpha_{i,m}(t) = \frac{o}{\sqrt{x^2+y^2}}$
8: $\beta_{i,m}(t) = \frac{l}{\sqrt{x^2+y^2}}$
9: $q_{i,m} = \begin{bmatrix} \alpha_{i,m}(t) \\ \beta_{i,m}(t) \end{bmatrix}$
10: $\vec{Q}_i(t) = \vec{Q}_i(t) \cup q_{i,m}$
11: **end for**
12: $\vec{Q}_{POP}(t) = \vec{Q}_{POP}(t) \cup \vec{Q}_i(t)$
13: **end for**
14: **return** $\vec{Q}_{POP}(t)$ /*Returns the quantum population*/

Illustration 6.1. Let us assume a system with six tasks ($J = 6$) and 3 CUs that include an SMD and two ESs, i.e., $\mathbb{R} = 2$. Initially, the six tasks will be assigned to any of these CUs. Here, $Bits = \lceil \log_2(\mathbb{R} + 1) \rceil = \lceil \log_2 3 \rceil = 2$ and thereby D is equal to $J \times Bits = 12$. Now, we have to generate the QAs of dimension 12. A QA is randomly generated using the Algorithm 1 and is shown in Table 3.

Lemma 6.1. *The time complexity of QuantumPopulation(N_P, D) is* $\Theta(N_P \times D)$.

Proof. A QA is generated in $\Theta(D)$ times (line 4-11, Algorithm 1). Hence, a population of size N_P can be generated in $\Theta(N_P \times D)$ time. □

6.2. Measuring quantum population

The quantum agent $\vec{Q}_i(t)$ is measured to produce a binary agent $\vec{B}_i(t) = \{b_{i,1}(t), b_{i,2}(t), \ldots, b_{i,D}(t)\}$, $b_{i,m} \in \{0, 1\}$. Algorithm 2 shows the measurement operation.

Illustration 6.2. In Illustration 6.1, we have a QA as shown in Table 3. For every $q_{i,m}(t)$, a random number ($rand[0, 1]$) is generated to observe the corresponding $b_{i,m}(t)$. $b_{i,m}(t)$ is observed as 0 if $rand[0, 1] \leq |\alpha_{i,m}|^2$, and 1 otherwise. The corresponding binary agent $\vec{B}_1(0)$ as shown in Table 3 is measured using Algorithm 2.

Lemma 6.2. *The time complexity of Measure($\vec{Q}_i(t)$) is* $\Theta(D)$.

Proof. Each qubit is measured for its corresponding binary bit as per the Algorithm 2. As, the dimension of the quantum agent is D, the time complexity is $\Theta(D)$. □

Algorithm 2 Measure($\vec{Q}_i(t)$)

Input: Quantum agent, $\vec{Q}_i(t) = \left[\begin{array}{c|c|c|c|c} \alpha_{i,1}(t) & \dots & \alpha_{i,m}(t) & \dots & \alpha_{i,D}(t) \\ \beta_{i,1}(t) & \dots & \beta_{i,m}(t) & \dots & \beta_{i,D}(t) \end{array}\right]$

Output: Observed binary agent, $\vec{B}_i(t) = \{b_{i,1}(t), b_{i,2}(t), \dots, b_{i,D}(t)\}$ of same dimension.

1: Initialize $\vec{B}_i(t) = \{\}$
2: **for** $m = 1$ to D **do**
3: **if** $rand[0,1] \leq |\alpha_{i,m}|^2$ **then**
4: $b_{i,m}(t) = 0$
5: **else**
6: $b_{i,m}(t) = 1$
7: **end if**
8: $\vec{B}_i(t) = \vec{B}_i(t) \cup b_{i,m}(t)$
9: **end for**
10: **return** $\vec{B}_i(t)$ /*Returns the corresponding binary agent*/

Table 3: A quantum agent (QA), binary agent, and decoded agent for Illustration 6.1, 6.2, and 6.3

Task List		Task 1		Task 2		Task 3		Task 4		Task 5		Task 6	
QA	$\vec{Q}_i(0)$	$q_{i,1}(0)$	$q_{i,2}(0)$	$q_{i,3}(0)$	$q_{i,4}(0)$	$q_{i,5}(0)$	$q_{i,6}(0)$	$q_{i,7}(0)$	$q_{i,8}(0)$	$q_{i,9}(0)$	$q_{i,10}(0)$	$q_{i,11}(0)$	$q_{i,12}(0)$
	$\alpha_i(0)$	-0.5	0.7071	0.5	0.866	-0.7071	-0.866	-0.5	-0.5	0.5	-0.7071	0.866	-0.866
	$\beta_i(0)$	0.866	-0.7071	-0.866	-0.5	-0.7071	-0.5	-0.866	-0.866	0.866	0.7071	0.5	0.5
Binary agent	$\vec{B}_i(0)$	$b_{i,1}(0)$	$b_{i,2}(0)$	$b_{i,3}(0)$	$b_{i,4}(0)$	$b_{i,5}(0)$	$b_{i,6}(0)$	$b_{i,7}(0)$	$b_{i,8}(0)$	$b_{i,9}(0)$	$b_{i,10}(0)$	$b_{i,11}(0)$	$b_{i,12}(0)$
		1	1	1	0	1	0	1	1	1	1	0	0
Decoded agent	$\vec{A}_1(0)$	$a_{i,1}(0)$		$a_{i,2}(0)$		$a_{i,3}(0)$		$a_{i,4}(0)$		$a_{i,5}(0)$		$a_{i,6}(0)$	
	Before Hashing	3		2		2		3		3		0	
	After Hashing	1		2		2		1		1		0	

6.3. Decoding of Agent

A binary agent $\vec{B}_i(t)$ of dimension D is broken into J number of pieces of length $Bits$ of each piece (as $D = J \times Bits$). Then each piece of $Bits$-length binary string is decoded to its equivalent decimal value to generate the decoded agent $\vec{A}_i(t) = \{a_{i,1}(t), a_{i,2}(t), ..., a_{i,d}(t), ..., a_{i,J}(t)\}$. Here, $a_{i,1}(t)$ is the decimal value of initial $Bits$-length binary string, $[b_{i,1}(t), ..., b_{i,Bits}(t)]_2$. Therefore, $a_{i,d}(t)$ is the decimal representation of the binary string $[b_{i,[(d-1)\times Bits+1]}(t), ..., b_{i,[d\times Bits]}(t)]_2$.

Note that the $a_{i,d}(t) \in [0, 2^{Bits} - 1]$ denotes the CU, where the task τ_d is assigned. $a_{i,d}(t) = 0$ indicates that τ_d is locally executed in the CU ϱ_0, i.e., the SMD s_d. $a_{i,d}(t) \in \{1, ..., K\}$ indicates that τ_d is offloaded to the UAV u_z, where $z = a_{i,d}(t)$. $a_{i,d}(t) \in \{K+1, ..., \mathbb{R}\}$ indicates that τ_d is offloaded to the ES e_x, where $x = (a_{i,d}(t) - K)$. If $a_{i,d}(t) > \mathbb{R}$, then the linear-hashing technique is used to map the value of $a_{i,d}(t)$ within the valid range $[0, \mathbb{R}]$ as per Algorithm 3.

Illustration 6.3. Consider the example of Illustrations 6.1 and 6.2. As $Bits = \lceil \log_2(\mathbb{R}+1) \rceil = 2$, each two-bit binary string is decoded to the equivalent decimal number. Therefore, $a_{i,1}(0) = 3$ is decoded from the binary string $[b_{i,1}(0), b_{i,2}(0)] = [11]_2$. Similarly, $a_{i,2}(0) = 2$ is decoded from $[b_{i,3}, b_{i,4}] = [10]_2$. As, $\mathbb{R} = 2$, the valid set of CUs is $\{0, 1, 2\}$. Therefore, $a_{i,1}(0) = 3$ is linearly hashed to $a_{i,1}(0) = 1$ as shown in line no. 9 of Algorithm 3. Thus, τ_1 is offloaded to the CU ϱ_1. As $a_{i,2}(0) = 2$, the τ_2 is offloaded to the CU ϱ_2. The complete decoded agent before-hashing as well as after-hashing are also shown in Table 3.

Lemma 6.3. *The time complexity of Decode($\vec{B}_i(t)$) is* $\Theta(D)$.

Proof. The line no. 4-17 of Algorithm 3 iterates for D times. The hashing of line no. 9 takes $\Theta(1)$ time. Hence, the time complexity is $\Theta(D)$. □

Algorithm 3 Decode($\vec{B}_i(t)$)

Input: A binary agent, $\vec{B}_i(t) = \{b_{i,1}(t), b_{i,2}(t), \ldots, b_{i,D}(t)\}$
Output: Decoded agent, $\vec{A}_i(t) = \{a_{i,1}(t), a_{i,2}(t), ..., a_{i,J}(t)\}$

Initialize $\vec{A}_i(t) = \{a_{i,1}(t), a_{i,2}(t), ..., a_{i,J}(t)\}$.
Initialize $Base = 2^{Bits-1}$, $d = 1, j = 1$.
Initialize $a_{i,j}(t) = 0$
while $d \leq D$ **do**
    $a_{i,j}(t) = a_{i,j}(t) + b_{i,d}(t) \times Base$
    $Base = Base/2$
    if $(d\%Bits) == 0$ **then**
        if $a_{i,j}(t) > \mathbb{R}$ **then**
            $a_{i,j}(t) = (a_{i,j}(t)) \mod \mathbb{R}$ /*Hashing*/
        end if
        $\vec{A}_i(t) = \vec{A}_i(t) \cup a_{i,j}(t)$
        $Base = 2^{Bits-1}$
        $j = j + 1$
        $a_{i,j}(t) = 0$
    end if
    $d = d + 1$
end while
return $\vec{A}_i(t)$ /*Returns the corresponding decoded agent*/

6.4. Fitness

The fitness value represents the quality of a decoded agent $\vec{A}_i(t) = \{a_{i,1}(t), a_{i,2}(t), ..., a_{i,J}(t)\}$. This is to remind again that $a_{i,d}(t)$ denotes the CU where the task τ_d is accomplished. The objective values are derived from a decoded agent $\vec{A}_i(t)$ as follows:

- *Objective 1 (minimize delay):* Each component ($a_{i,d}(t)$) of the agent represents the CU where the task τ_d is offloaded. The time (δ^d_{Total}) required to complete τ_d can be calculated using eq. (2). The maximum delay (δ_{total}) of the system can be calculated using eq. (4).

  Let us assume that all the J number of tasks are offloaded serially to the first UAV (i.e., ϱ_1). Then the time (δ_{max}) required to serially execute all the tasks at ϱ_1 is as per the eq. (35).

$$\delta_{max} = \sum_{d=1}^{J} \{\delta_t^{d1} + \delta_r^{1d} + \delta_c^{1d}\} \tag{35}$$

  Without loss of generality, it can be claimed that $\delta_{max} > \delta_{total}$. Now, the total delay can be normalized within the range of (0,1) as $\delta_{total} = \frac{\delta_{total}}{\delta_{max}}$.

  Note that the τ_d has a hard deadline of δ^d_{thr} by the time it should be finished. If one or more tasks of the agent $\vec{A}_i(t)$ violate the deadline, then a penalty is imposed as follows:

$$\delta_{total} = \begin{cases} 1, & \text{if } (\exists a_{i,d}(t), s.t., \delta^d_{Total} > \delta^d_{thr}) \\ \delta_{total}, & \text{Otherwise.} \end{cases} \tag{36}$$

  The first objective is to minimize δ_{total}.

- *Objective 2 (minimize energy):* Energy consumption of individual SMD and UAV can be calculated using eqs. (7) and (13) respectively. The total energy consumption (E_{total}) of the system can be calculated as per the eq. (14).

  As mentioned above, if all the J number of tasks are offloaded serially to the first UAV (i.e., ϱ_1) and also executed locally, then the required energy (E_{max}) to execute all the tasks locally as well as at ϱ_1 is as per the eq. (37).

$$E_{max} = \sum_{b=1}^{K} E^b_{prop} + \sum_{d=1}^{J} \{E^{d1}_t + E^{d1}_r + E^{1d}_r + E^{1d}_t + E^{1d}_c\} + \sum_{d=1}^{J} E^{0d}_c \tag{37}$$

  Without loss of generality, it can be claimed that $E_{max} > E_{total}$. Now, the total energy can be normalized within the range of (0,1) as $E_{total} = \frac{E_{total}}{E_{max}}$.

  It is important to note that the SMDs and UAVs come with limited energy. Therefore, the total energy consumption of an SMD (E^d_{SMD}) and a UAV (E^b_{UAV}) should be limited by their corresponding battery limit E^{thr}_d and $E_{uav-thr}$ respectively. If the battery limit exceeds for one or more SMDs or UAVs of the agent $\vec{A}_i(t)$, then a penalty is imposed as follows:

$$E_{total} = \begin{cases} 1, & \text{if } (\exists s_d, s.t., E^d_{SMD} > E^d_{thr}) \text{ OR } (\exists b \in \{1...K\}, s.t., E^b_{UAV} > E^b_{uav-thr}) \\ E_{total}, & \text{Otherwise.} \end{cases} \tag{38}$$

  The second objective is to minimize E_{total}.

- *Objective 3 (minimize price):* The total price (P_{total}) that needs to be paid by all the SMDs is calculated by using the eq. (15). The maximum price (P_{max}) may need to be paid when the energy consumption is maximum as discussed above. It can be estimated as:

$$P_{max} = \sum_{d=1}^{J} \{\mathbb{P} \times B_d\} + \{w1.E_{max}\} + \{w2.E_{max}\} \tag{39}$$

  Without loss of generality, it can be claimed that $P_{max} > P_{total}$. Now, the total price can be normalized within the range of (0,1) as $P_{total} = \frac{P_{total}}{P_{max}}$. The third objective is to minimize P_{total}.

- *Objective 4 (maximize resource utilization):* In the ideal case, all the CUs should be used maximally. The resource utilization ($U_R(Total)$) of the system is calculated using eq. (5). Here the fourth objective is to maximize $U_R(Total)$.

Weight sum approach (WSA) (Kuila and Jana, 2014) merges the four objectives after updating the individual objectives as per the penalty and scaling model. The fitness calculation is shown in eq. (40).

$$\text{Minimize } fitness = (\omega_1 \times \delta_{total}) + (\omega_2 \times E_{total}) + (\omega_3 \times P_{total}) + (\omega_4 \times (1 - U_R(Total))) \tag{40}$$

The weight values of the associated objectives are ω_1, ω_2, ω_3, and ω_4, such that $\sum_{j=1}^{4} \omega_j = 1$ and $\forall \omega_j, 0 \leq \omega_j \leq 1$. The most efficient agent is the agent with the least fitness value. The Algorithm 4 performs the fitness computation.

Algorithm 4 : Fitness($\vec{A}_i(t)$)

Input: An agent, $\vec{A}_i(t) = \{a_{i,1}(t), a_{i,2}(t), ..., a_{i,d}(t), ..., a_{i,J}(t)\}$ and $\mathbb{R} = Y + K$.
Output: Fitness value for $\vec{A}_i(t)$.

Initialize $flag_1 = flag_2 = 0, \omega_1 = 0.25, \omega_2 = 0.25, \omega_3 = 0.25, \omega_4 = 0.25$.
Initialize $E_{total} = P_{total} = 0$, $T\delta^b_{CU} = E^b_{prop} = 0, \forall b \in \{1, \mathbb{R}\}$.
for $d = 1$ to J
  if $(a_{i,d}(t) == 0)$ **then** //Executed locally.
    $T\delta^d_{SMD} = \delta^d_{Total} = \delta^{0d}_c$
    Compute $ETotal_d = E^{0d}_c$. //Sec. 3.4.
    $E_{total} = E_{total} + ETotal_d$
  else //Offloaded to ES or UAV.
    Initialize $b = a_{i,d}(t)$
    $\delta^d_{Total} = \delta^{db}_t + \delta^{bd}_r + \delta^{bd}_c$
    $T\delta^b_{CU} = T\delta^b_{CU} + \delta^d_{Total}$ //from eq. (3).
    Compute $ETotal_d = E^{db}_t + E^{bd}_r + E^{bd}_c$. //Using eq. (6).
    if $(1 \leq a_{i,d}(t) \leq K)$ **then** //Offloaded to UAV.
      if $(E^b_{prop} == 0)$ **then**
        Calculate E^b_{prop} //Using eq. (12).
        $E^b_{UAV} = E^b_{UAV} + E^b_{prop} + \{E^{db}_t + E^{bd}_r + E^{bd}_c\}$ //Using eq. (13).
        $E_{total} = E_{total} + E^b_{prop} + ETotal_d$
      else
        $E^b_{UAV} = E^b_{UAV} + ETotal_d$
        $E_{total} = E_{total} + ETotal_d$
      end if
      if $(E^b_{UAV} \geq E_{uav-thr})$, **then** $flag_2 + +$.
      $P_{total} = P_{total} + w_2 \times (E^{bd}_r + E^{bd}_c)$ //Price for UAV.
    else
      $E_{total} = E_{total} + ETotal_d$
      $P_{total} = P_{total} + w_1 \times (E^{bd}_r + E^{bd}_c)$ //Price for edge.
    end if
  end if
  $P_{total} = P_{total} + \{B(d) \times \mathbb{P}\}$ //Price for SMD bandwidth.
  if $(\delta^d_{Total} \geq \delta^d_{thr})$, **then** $flag_1 + +$.
end for
$\delta_{total} = \max\{T\delta^d_{SMD}, T\delta^b_{CU} | \forall b, 1 \leq b \leq \mathbb{R}, \forall d, 1 \leq d \leq J\}$ //Using eq. (4).
Calculate δ_{max}, E_{max} and P_{max} //Using eq. (35), (37), and (39).
Compute $U_R(Total) = \frac{1}{J+\mathbb{R}}\left\{\sum_{d=1}^{J} \frac{T\delta^d_{SMD}}{\delta_{total}} + \sum_{b=1}^{\mathbb{R}} \frac{T\delta^b_{CU}}{\delta_{total}}\right\}$ //Using eq. (5).
Normalize $\delta_{total} = \frac{\delta_{total}}{\delta_{max}}$, $E_{total} = \frac{E_{total}}{E_{max}}$, $P_{total} = \frac{P_{total}}{P_{max}}$
if $(flag_1 > 0)$, **then** $\delta_{total} = 1$. //Penalty as maximum delay.
if $(flag_2 > 0)$, **then** $E_{total} = 1$. //Penalty as maximum energy consumption.
$fitness = (\omega_1 \times \delta_{total}) + (\omega_2 \times E_{total}) + (\omega_3 \times P_{total}) + (\omega_4 \times (1 - U_R(Total)))$. // eq. (40).
return $fitness$ /*Returns the corresponding fitness for the decoded agent*/

Remark 6.1. Three distinct penalties are introduced in this study to obtain a better agent or solution. First, each τ_d has a hard deadline of δ^d_{thr}. If a task fails such a deadline then the penalty is introduced as in eq. (36). Next, the SMDs and UAVs are energy-constrained, and therefore, the

total energy consumption of the SMDs and UAVs must be limited by their corresponding threshold energy. If the offloading decision by an agent fails such restrictions then a penalty is invoked by the eq. (38).

Remark 6.2. Note that different magnitudes of the objective parameters, $\delta_{total}, E_{total}, P_{total}$, and $U_R(Total)$ could result in giving favor to the parameters with higher value. To ensure fair preference, normalization is applied to all the above objective parameters within the range of (0,1).

Lemma 6.4. *The time complexity of Fitness($\vec{A}_i(t)$) is* $\Theta(J)$.

Proof. The line no. 3-31 of Algorithm 4 iterates for J times. The maximum delay, energy, and price are shown in line no. 33 is calculated in $\Theta(1)$ time. Similarly, resource utilization calculation (line no. 34), scaling operation (line no. 35), and fitness calculation (line no. 38) is done in $\Theta(1)$ time. Hence, the time complexity is $\Theta(J)$. □

6.5. Mass and Applied Force

Each object in the universe is attracted by other objects based on its mass and distance, as per Newton's Laws of Gravity. An agent $\vec{B}_i(t)$ meets the gravitational forces from all other agents $\vec{B}_j(t)$ in the population by imitating the laws of gravity. This force is inversely proportional to the Euclidean distance between the agents and is directly proportional to the product of their masses. The mass $M_i(t)$ of $\vec{A}_i(t)$ can be calculated using eq. (28). Note that $\vec{A}_i(t)$ is the corresponding decoded agent of $\vec{B}_i(t)$ and the fitness can be computed from the decoded agent $\vec{A}_i(t)$ only. Let $f_{i,j,m}(t)$ represents the gravitational force between $\vec{B}_i(t)$ and $\vec{B}_j(t)$ towards the m^{th} component as given in eq. (41).

$$f_{i,j,m}(t+1) = G(t) \times \frac{M_i(t) \times M_j(t)}{R_{i,j} + \measuredangle} \times (b_{j,m}(t) - b_{i,m}(t)) \tag{41}$$

where, $R_{i,j} = ||\vec{B}_i(t), \vec{B}_j(t)||$ is the Euclidean distance between $\vec{B}_i(t)$ and $\vec{B}_j(t)$. $\measuredangle$ is a tiny positive constant and $G(t)$ is the gravitational constant as discussed in the sec. 5.1. The total force $(F_{i,m}(t+1))$ on the agent $\vec{B}_i(t)$ towards the m^{th} component is calculated using eq. (30). Let us assume $\vec{\mathbf{F}}_i(t+1) = \{F_{i,1}(t+1), F_{i,2}(t+1), ..., F_{i,D}(t+1)\}$ be the total force on $\vec{B}_i(t+1)$. The Algorithm 5 is used to compute the force.

Lemma 6.5. *Force($\vec{B}_i(t)$) has the time complexity of* $\Theta(D \times N_P)$.

Proof. Each agent in the population is drawn by the gravitational force exerted by all other agents towards all the D components. The population size is N_P. Therefore, the time complexity of Force($\vec{B}_i(t)$) is $\Theta(D \times N_P)$. □

6.6. Computation of Acceleration, Angular Velocity, and Quantum Population

The acceleration $(ac_{i,m}(t+1))$ of m^{th} component $(a_{i,m}(t))$ of $\vec{A}_i(t)$ can be calculated using eq. (31). A qubit can be imagined as a point on the surface of the Bloch sphere. Therefore, a small change in the position of a qubit can also be denoted as a change in the angle of the sphere. The angular velocity (ω) is calculated as $\omega = \frac{v}{r}$. r is the radius of the sphere. It is assumed that $r = 1$ and therefore, $\omega = v$. For every component of a QA, there will be an angular velocity and a rotational angle or angular displacement associated with it. The angular velocity of a QA $\vec{Q}_i(t)$ can be represented as $\vec{\mathbf{V}}_i(t) = \{\omega_{i,1}(t), ..., \omega_{i,m}(t), ..., \omega_{i,D}(t)\}$ and $\omega_{i,m}(t)$ can be updated as:

$$\omega_{i,m}(t+1) = rand[0,1] \times \omega_{i,m}(t) + ac_{i,m}(t+1) \tag{42}$$

Algorithm 5 Force($\vec{B}_i(t)$)

Input: An agent, $\vec{B}_i(t) = \{b_{i,1}(t), b_{i,2}(t), ..., b_{i,m}(t), ..., b_{i,D}(t)\}$
Output: Overall force on $\vec{B}_i(t)$: $\vec{\mathbf{F}}_i(t+1) = \{F_{i,1}(t+1), ..., F_{i,m}(t+1), ..., F_{i,D}(t+1)\}$

1: Initialize $\vec{\mathbf{F}}_i(t+1) = \{F_{i,1}(t+1), ..., F_{i,m}(t+1), ..., F_{i,D}(t+1)\}$.
2: **for** $m = 1$ to D
3: $\quad F_{i,m}(t+1) = 0$
4: $\quad$ **for** $j = 1$ to N_P
5: $\quad\quad$ **if** $j \neq i$ **then**
6: $\quad\quad\quad f_{i,j,m}(t+1) = G(t) \times \dfrac{M_i(t) \times M_j(t)}{R_{i,j} + \nearrow} \times (b_{j,m}(t) - b_{i,m}(t))$ // from eq. (41).
7: $\quad\quad\quad F_{i,m}(t+1) = F_{i,m}(t+1) + rand_j \times f_{i,j,m}(t+1)$ // from eq. (30).
8: $\quad\quad$ **end if**
9: $\quad$ **end for**
10: **end for**
11: **return** $\vec{\mathbf{F}}_i(t+1)$ /*Returns the corresponding force*/

where $rand[0,1]$ is the random number and initially (i.e., $t = 0$) all $\omega_{i,m}(0) = 0$. The angular displacement or the rotation angle ($\Delta\theta_{i,m}(t)$) is calculated in $\Delta\tau$ time as $\Delta\theta_{i,m}(t) = \omega_{i,m}(t) \times \Delta\tau$. For a discrete system, $\Delta\tau$ is one. So, angular displacement ($\Delta\theta_{i,m}(t)$) is equal to angular velocity ($\omega_{i,m}(t)$). The $\Delta\theta_{i,m}(t)$ is calculated with respect to the available quadrant as shown in eq. (43).

$$\Delta\theta_{i,m}(t) = \begin{cases} \omega_{i,m}(t), & \text{if } (\alpha_{i,m}(t) \times \beta_{i,m}(t) \geq 0) \\ -\omega_{i,m}(t), & \text{otherwise} \end{cases} \tag{43}$$

Now, each qubit of a QA is updated using the quantum rotation gate as follows:

$$\begin{bmatrix} \alpha_{i,m}(t+1) \\ \beta_{i,m}(t+1) \end{bmatrix} = \begin{bmatrix} \cos(\Delta\theta_{i,m}(t)) & -\sin(\Delta\theta_{i,m}(t)) \\ \sin(\Delta\theta_{i,m}(t)) & \cos(\Delta\theta_{i,m}(t)) \end{bmatrix} \begin{bmatrix} \alpha_{i,m}(t) \\ \beta_{i,m}(t) \end{bmatrix} \tag{44}$$

The Algorithm 6 provides the angular velocity, acceleration, and updation of a QA $\vec{Q}_i(t-1)$. Algorithm 7 demonstrates the overall process of QGSA.

Lemma 6.6. *The time complexity of UpdAngVelQPop($\vec{Q}_i(t-1)$, $\vec{\boldsymbol{V}}_i(t-1)$, $\vec{\boldsymbol{F}}_i(t)$) is* $\Theta(D)$.

Proof. The line no. 1 - 11 iterates for $\Theta(D)$ times. The $\alpha_{i,m}(t)$ and $\beta_{i,m}(t)$ are updated in $\Theta(1)$ time in line no. 9 - 10. Hence, the time complexity of UpdAngVelQPop($\vec{Q}_i(t-1), \vec{\mathbf{V}}_i(t-1), \vec{\mathbf{F}}_i(t)$) is $\Theta(D)$. □

Lemma 6.7. *The time complexity of QGSA($D, J, \mathbb{R}, N_P$) is* $\Theta(N_P^2 \times MAX \times D)$.

Proof. The time complexity of QuantumPopulation(N_P, D) is $\Theta(N_P \times D)$ (refer to Lemma 6.1). The time complexity of Measure($\vec{Q}_i(t)$) is $\Theta(D)$ (refer to Lemma 6.2). The Decode($\vec{B}_i(t)$) takes $\Theta(D)$ time (refer to Lemma 6.3). As per Lemma 6.4, time complexity of Fitness($\vec{A}_i(t)$) is $\Theta(J)$. The time complexity of Force($\vec{B}_i(t)$) is $\Theta(D \times N_P)$. (refer to Lemma 6.5). The UpdAngVelQPop($\vec{Q}_{POP}(t-1), \vec{\mathbf{V}}_i(t-1), \vec{\mathbf{F}}_i(t), D$) takes $\Theta(D)$ (refer to Lemma 6.6). Out of all the above functions, the QuantumPopulation(N_P, D) is executed once (line 2, Algorithm 7). All the remaining functions can be executed maximum $\Theta(N_P \times MAX)$ times. Therefore, the overall time complexity of QGSA($D, J, \mathbb{R}, N_P$) is $\Theta(N_P^2 \times MAX \times D)$ as shown in Table 4. □

Algorithm 6 UpdAngVelQPop($\vec{Q}_i(t-1), \vec{\mathbf{V}}_i(t-1), \vec{\mathbf{F}}_i(t)$)

Input: (1) $\vec{Q}_i(t-1)$: Previous quantum agent.
(2) $\vec{\mathbf{V}}_i(t-1) = \{\omega_{i,1}(t-1), ..., \omega_{i,m}(t-1), ..., \omega_{i,D}(t-1)\}$: Previous angular velocity.
(3) $\vec{\mathbf{F}}_i(t) = \{F_{i,1}(t), F_{i,2}(t), ..., F_{i,D}(t)\}$: Updated force

Output: Updated angular velocity, $\vec{\mathbf{V}}_i(t)$ and $\vec{Q}_i(t)$.

for $m = 1$ to D
  $ac_{i,m}(t) = \frac{F_{i,m}(t)}{M_i(t)}$ // from eq. (31).
  $\omega_{i,m}(t) = rand \times \omega_{i,m}(t-1) + ac_{i,m}(t)$ // from eq. (42).
  if $(\alpha_{i,m}(t-1) \times \beta_{i,m}(t-1)) \geq 0$ **then**
    $\Delta\theta_{i,m}(t) = \omega_{i,m}(t)$ // from eq. (43).
  else
    $\Delta\theta_{i,m}(t) = -\omega_{i,m}(t)$ // from eq. (43).
  end if
  $\alpha_{i,m}(t) = (\cos(\Delta\theta_{i,m}(t)) \times \alpha_{i,m}(t-1)) + (\sin(\Delta\theta_{i,m}(t)) \times \beta_{i,m}(t-1))$
  $\beta_{i,m}(t) = (-\sin(\Delta\theta_{i,m}(t)) \times \alpha_{i,m}(t-1)) + (\cos(\Delta\theta_{i,m}(t)) \times \beta_{i,m}(t-1))$
end for
return $\{\vec{Q}_i(t), \vec{\mathbf{V}}_i(t)\}$ /*Returns the QA and angular velocity*/

Table 4: Time complexity analysis of different phases in QGSA (Lemma: 6.7)

Module	Complexity	Frequency	Total complexity
QuantumPopulation(N_P, D)	$\Theta(N_P \times D)$	1	$\Theta(N_P \times D)$
Measure($\vec{Q}_i(t)$)	$\Theta(D)$	$\Theta(N_P \times MAX)$	$\Theta(N_P \times MAX \times D)$
Decode($\vec{B}_i(t)$)	$\Theta(D)$	$\Theta(N_P \times MAX)$	$\Theta(N_P \times MAX \times D)$
Fitness($\vec{A}_i(t)$)	$\Theta(J)$	$\Theta(N_P \times MAX)$	$\Theta(N_P \times MAX \times J)$
Force($\vec{B}_i(t)$)	$\Theta(N_P \times D)$	$\Theta(N_P \times MAX)$	$\Theta(N_P^2 \times MAX \times D)$
UpdAngVelQPop($\vec{Q}_i(t-1), \vec{\mathbf{V}}_i(t-1), \vec{\mathbf{F}}_i(t)$)	$\Theta(D)$	$\Theta(N_P \times MAX)$	$\Theta(N_P \times MAX \times D)$

7. Simulation Results

7.1. System Parameters and Simulation Configuration

The research is replicated on a computer with Windows 10, Intel (R) CoreTM i3-1005G1 CPU running at 1.19 GHz, and 4 GB of RAM. When simulating the scenarios, Python 3.8 was utilized. Three different edge-computing scenarios are considered as discussed in the subsections 7.2, 7.3, and 7.4 to measure the performance of the proposed QGSA.

The research conducted by Sun: DE (Sun et al., 2021), You: PSO (You and Tang, 2021), Pham: WOA (Huynh et al., 2020), and Tang: DRL (Tang and Wong, 2020) are replicated in scenario 2 (subsection 7.3) and scenario 3 (subsection 7.4). Moreover, the conventional GA (Ram and Kuila, 2023) and GSA (Ghosh and Kuila, 2023) are constructed using the proposed fitness function for scenarios 2 and 3 with the population size, crossover probability, mutation rate, and the maximum number of iterations as 100, 0.7, 0.5, and 100, respectively. GA is a population-based EA where a solution is represented as a chromosome. The GA operators, selection, crossover, and mutation are applied on each chromosome of the population to enhance the quality of the solution. A crossover between two parent chromosomes generates two child chromosomes (offspring). The offspring are further mutated with an expectation of improvement. A child chromosome replaces its parent in the next generation if found to be better in fitness. The process is iterated till the termination condition is met. The GSA is discussed in the subsection 5.1.

Algorithm 7 : QGSA$(D, J, \mathbb{R}, N_P)$

Input: (1) J: Dimension of the agents, (2) N_P: Population size, (3) $\mathbb{R}$: Number of servers or CUs.
Output: Best agent of the population.

Initialize $t = 0$.
$\vec{Q}_{POP}(t)$ = QuantumPopulation(N_P, D). // Algorithm 1.
for $i = 1$ to N_P
    $\vec{B}_i(t)$ = Measure$(\vec{Q}_i(t))$. // Algorithm 2.
    $\vec{A}_i(t)$ = Decode$(\vec{B}_i(t))$. // Algorithm 3.
    $fitness_i(t)$=Fitness$(\vec{A}_i(t))$. // Algorithm 4.
end for
$best = min\{fitness_i(t)|\forall i, 1 \leq i \leq N_P\}$. // Minimization problem.
$worst = max\{fitness_i(t)|\forall i, 1 \leq i \leq N_P\}$.
Compute $M_i(t), \forall \vec{A}_i(t), 1 \leq i \leq N_P$ // Using eq. (28).
Initialize $\vec{\mathbf{V}}_i(t)$ and $\vec{\mathbf{F}}_i(t), \forall \vec{Q}_i(t) \in \vec{Q}_{POP}$ // $\omega_{i,1}(t) = F_{i,m}(t) = 0, \forall m, 1 \leq m \leq D$.
for $t = 1$ to MAX
    Update $G(t) = G(0) \times e^{(\frac{-\mathbb{k} \times t}{MAX})}$.
    for $i = 1$ to N_P
        $\vec{\mathbf{F}}_i(t)$=Force$(\vec{B}_i(t-1))$ // Algorithm 5.
        $\{\vec{Q}_i(t), \vec{\mathbf{V}}_i(t)\}$ =UpdAngVelQPop$(\vec{Q}_i(t-1), \vec{\mathbf{V}}_i(t-1), \vec{\mathbf{F}}_i(t))$ // Algorithm 6.
        $\vec{B}_i(t)$ = Measure$(\vec{Q}_i(t))$. // Algorithm 2.
        $\vec{A}_i(t)$ = Decode$(\vec{B}_i(t))$. // Algorithm 3.
        $fitness_i(t)$=Fitness$(\vec{A}_i(t))$. // Algorithm 4.
        if $best \geq fitness_i(t)$ **then** // Minimization problem.
            $best = fitness_i(t)$
        end if
        if $worst \leq fitness_i(t)$ **then** // Minimization problem.
            $worst = fitness_i(t)$
        end if
    end for
    Update $M_i(t)$ // Using eq. (28).
    Update $R_{i,j}$.
end for
$BestAgent$ = Argmin$\{fitness_i(t)|\forall \vec{A}_i(t), 1 \leq i \leq N_P\}$. // Minimization problem.
Return $BestAgent$. /*Returns the corresponding best agent among the decoded agents*/

Scenarios 2 and 3 are assumed to have varying numbers of tasks between 100 and 500. For QGSA, population size and the maximum number of iterations were taken as 50 and 100, respectively. Table 5 provides the values of the parameters (Zhang et al., 2023; Sun et al., 2021; Liao et al., 2023). From here onward, Sun: DE (Sun et al., 2021), You: PSO (You and Tang, 2021), Pham: WOA (Huynh et al., 2020), and Tang: DRL (Tang and Wong, 2020) will be marked as DE, PSO, WOA, DRL, respectively.

7.2. Scenario - 1

Two CUs are used in the scenario -1, which includes an ES (ϱ_1) and a rotary-wing UAV (ϱ_2). The SMD from where the task is generated is also available for local computation of the task. Let there be ten SMDs as $\{s_1, s_2, ..., s_{10}\}$. Each SMD generates one task. Therefore, the set of tasks can be represented as $\{\tau_1, \tau_2, ..., \tau_{10}\}$. All the task requests are sent to the Agent. For this scenario,

Table 5: Parameters and values

Notation	Values	Notation	Values
$\vec{B}_d$	[0-1000000] Hz	d_r	0.3
σ^2	-100 dBm	v_{tip}	120 m/sec
k	10^{-27}	v_0	4.03
D	4 - 200	S	0.05
N_P	10-30	ρ	1.225
MAX	100	C_d	1000 cycles/bit
$\mathbb{k}$	20	W_{in}, W_{out}	1 MB , 2 KB
$G(0)$	100	f_b, f_d	3-3.5 GHz, 0.5 GHz
P_t^d, P_t^b, P_r^d, P_r^b	0.2 W, 2 W, 0.1 W, 0.1 W	w_1, w_2	2, 5
P_0, P_1	158.76 W, 88.63 W	$\mathbb{P}$	0.45
P_2	11.46 W	$\omega_1, \omega_2, \omega_3, \omega_4$	0.25, 0.25, 0.25, 0.25

Table 6: Offloading 10 tasks in 2 CUs ($\{\varrho_1\}, \{\varrho_2\}$) and SMD using QGSA (subsection: 7.2)

Task:	τ_1	τ_2	τ_3	τ_4	τ_5	τ_6	τ_7	τ_8	τ_9	τ_{10}
Allocated CUs	ϱ_1	s_2	s_3	s_4	ϱ_1	ϱ_1	s_7	s_8	ϱ_2	ϱ_2

ϱ_1 and ϱ_2 have a computation capacity of 2, and 4 GHz respectively, and SMDs have 0.5 GHz. The task offloading decision is made using the proposed QGSA in the Agent. Table 6 shows the offloading of the tasks to the CUs. The tasks are either computed locally or offloaded to a CU.

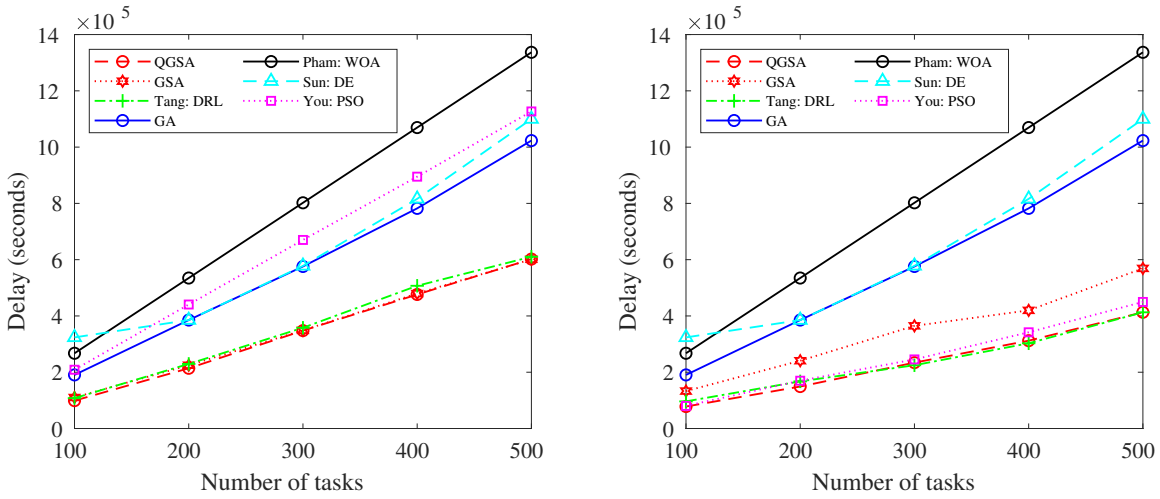


Fig. 4: Comparison based on the delay, (a) Scenario-2 and (b) Scenario-3.

7.3. Scenario - 2

Similar to scenario - 1, in this scenario, there are 2 CUs, where one is an ES (ϱ_1) and another is a rotary-wing UAV (ϱ_2). Along with that, SMDs are available for local computation of their tasks. Table 7 shows the fitness comparison between QGSA, GA, DE, PSO, WOA, GSA, and DRL. The corresponding delay, energy, price, and resource utilization values contributing to the

fitness value, are also calculated. The comparison is shown in Fig. 4(a), Fig. 5(a), Fig. 6(a), and Fig. 7(a) respectively. Fig. 8(a) is plotted for fitness value comparison. When compared to GA, DE, PSO, WOA, and DRL, QGSA has successfully decreased overall fitness, delay, and energy. Table 7 shows that GSA has reduced overall fitness but performed better in one instance than QGSA. In a similar vein, QGSA has reduced delay, energy, and cost while outperforming GSA in all but one case. By comparison, QGSA has successfully lowered prices than DRL, GA, and PSO. Regarding price reduction, WOA, and DE outperformed QGSA. When it came to utilization of resources, DRL, PSO, and DE performed better than QGSA. In contrast, QGSA performed better in resource utilization than GSA, GA, and WOA. It can be observed that the energy and delay appear to follow a similar pattern. This is because the energy consumption heavily depends on the delay.

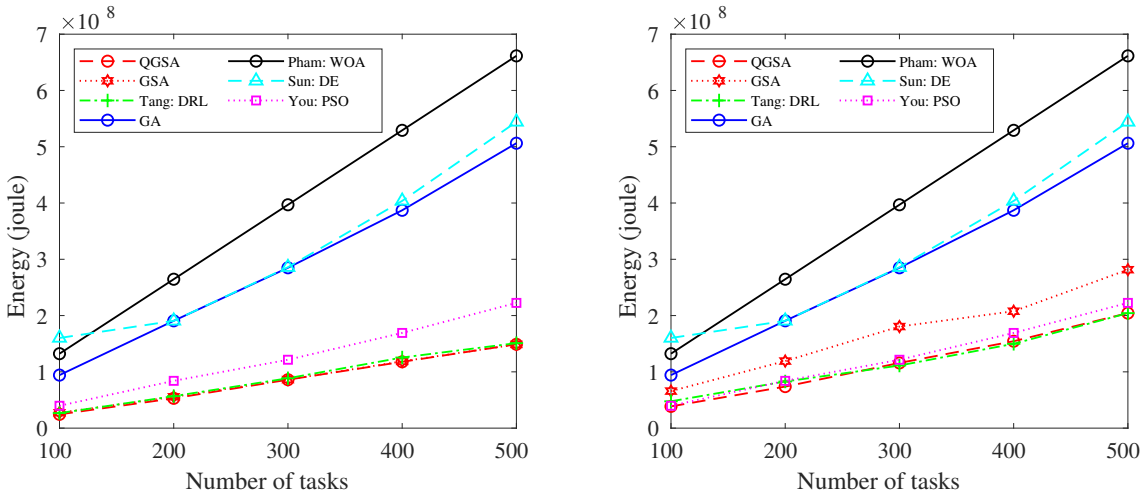


Fig. 5: Comparison based on the energy (a) Scenario-2, (b) Scenario-3.

Table 7: Comparison based on the fitness for scenario-2 (subsection: 7.3)

Amount of tasks	100	200	300	400	500
QGSA	0.462271	0.481707	0.501798	0.508799	0.511897
You: PSO (You and Tang, 2021)	0.67896	0.69292	0.69389	0.69941	0..70649
GA	0.84167	0.84626	0.84665	0.85273	0.86305
Sun: DE (Sun et al., 2021)	0.71465	0.75475	0.75335	0.77075	0.79216
Pham: WOA (Huynh et al., 2020)	0.83920	0.84056	0.84125	0.84143	0.84176
GSA	0.487602	0.494411	0.503034	0.511035	0.51189
Tang: DRL (Tang and Wong, 2020)	0.482465	0.49893	0.50934	0.525847	0.515640

7.4. Scenario - 3

In this scenario, it is considered that there are 4 CUs with two ES (ϱ_1, ϱ_2) and two rotary-wing UAVs (ϱ_3, ϱ_4). Along with CUs, SMDs are available for local computation of their tasks. QGSA was found to be better for maximum instances while minimizing delay (Fig. 4(b)), energy (Fig. 5(b)), and overall fitness (Fig. 8(b)) when compared with DE, GA, PSO, WOA, and GSA. Also,

QGSA maximized resource utilization (Fig. 7(b)) when compared with other algorithms. For pricing, it was found that QGSA gave better results than DRL, GA, and PSO. DE, GSA, and WOA gave better results than QGSA with respect to pricing as shown in Fig. 6(b). Table 7 shows the fitness comparison between QGSA, GA, DE, PSO, and WOA. As the aim of the research is to find a solution where the overall fitness would be minimal, QGSA has successfully performed better than other algorithms in scenarios 2 and 3. In addition, QGSA has done better than the compared algorithms for maximal objectives.

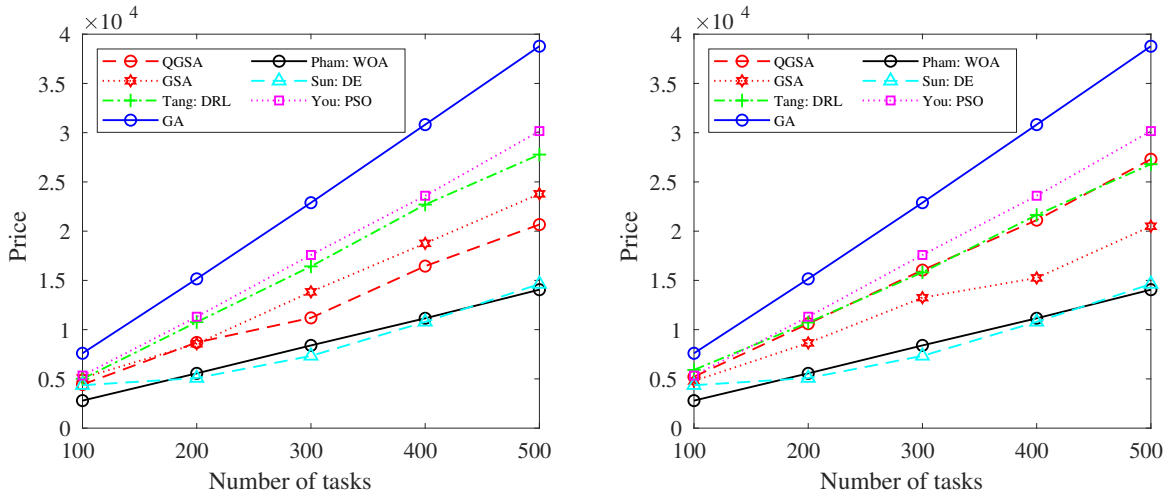


Fig. 6: Comparison based on the price, (a) Scenario-2 and (b) Scenario-3.

Table 8: Comparison based on the fitness for scenario-3 (subsection: 7.4)

Amount of tasks	100	200	300	400	500
QGSA	0.671251	0.673629	0.679439	0.679248	0.687589
PSO	0.67896	0.69292	0.69389	0.69941	0.70649
GA	0.84167	0.84626	0.84665	0.85273	0.86305
DE	0.71465	0.75475	0.75335	0.77075	0.79216
WOA	0.83920	0.84056	0.84125	0.84143	0.84176
GSA	0.728541	0.708492	0.711742	0.683153	0.697805
DRL	0.705625	0.685464	0.674986	0.679304	0.685900

7.5. Analysis of Variance(ANOVA)

The significance of the result is examined using ANOVA (Ghosh and Kuila, 2023). QGSA, GA, PSO, WOA, DE, GSA, and DRL are all evaluated in this study utilizing the null hypothesis (H_0) and alternative hypothesis (H_1). If the P value is less than the α threshold, which is 0.05, then H_0 is rejected. In addition, if F-static (F_{stat}) > F-critical (F_{cric}), then H_0 is rejected. In this case, H_0 denotes the equality of QGSA, GA, PSO, WOA, DE, GSA, and DRL means. According to H_1, they are not the same. Ten samples were obtained for QGSA, GA, PSO, WOA, DE, GSA, and DRL for each of the 100 tasks in the ANOVA test. Only the analysis for overall fitness based on scenario 2 is displayed here. The result of ANOVA is shown in Table 9.

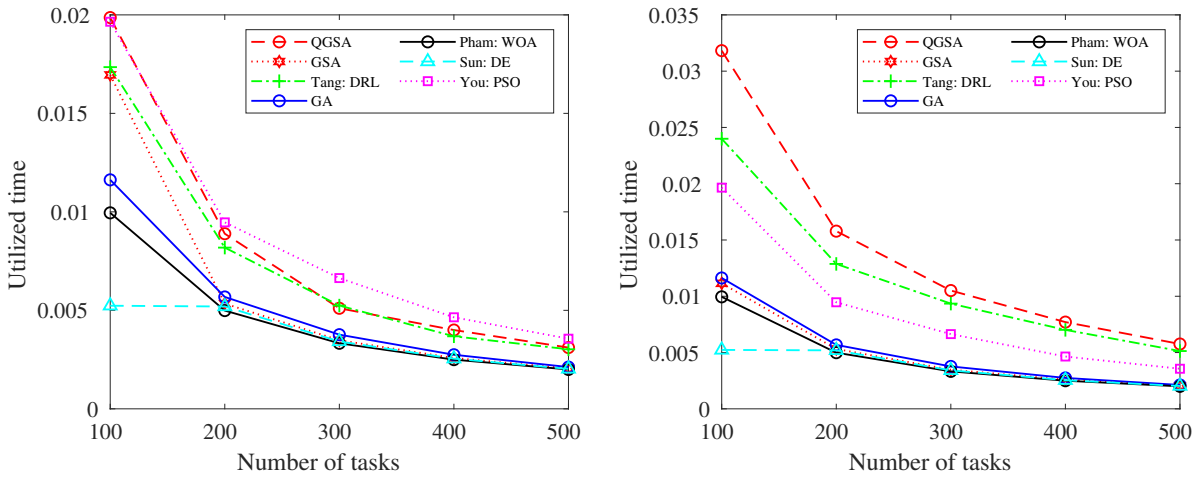


Fig. 7: Comparison based on the resource utilization, (a) Scenario-2 and (b) Scenario-3.

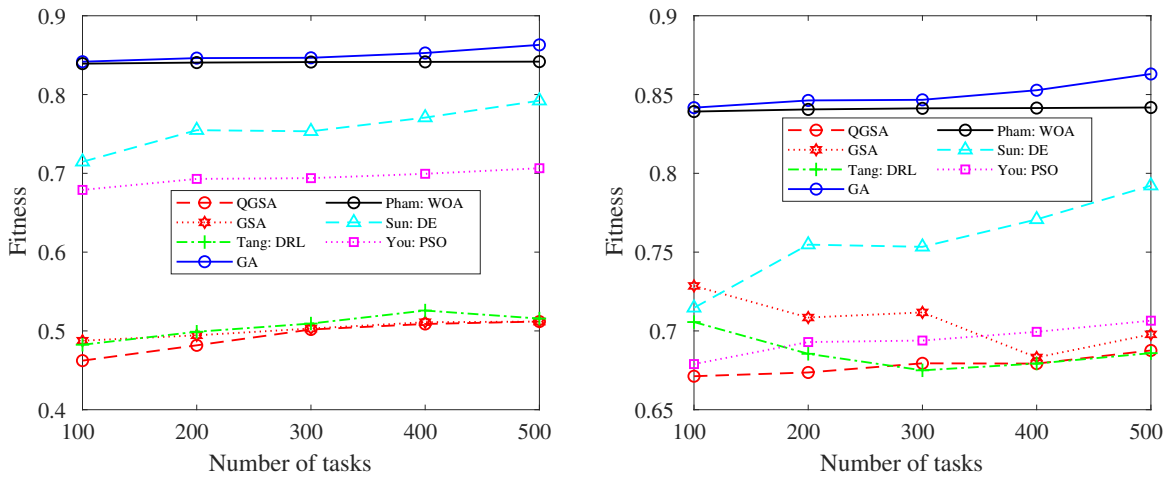


Fig. 8: Comparison based on the overall fitness, (a) Scenario-2 and (b) Scenario-3.

Table 9: Result of ANOVA test performed on scenario-2 with 100 tasks (subsection: 7.5)

Source of variations	Sum of Square	Degree of Freedom	Mean Square (MS)	F-statistic	p-value	F-critical
Between Groups	0.4571	6	0.07619	62.3195	0	2.323
Within Groups	0.05135	42	0.0012237			
Total	0.5085	48	0.01059			

Since the p-value$< \alpha$, H_0 is rejected in this case. It is believed that some groups' averages are

not equal. It indicates that the likelihood of a type 1 error—rejecting an accurate H_0 is low. The higher the support for H_1, the smaller the p-value. The F-statistic, 62.3195, does not fall inside the 95% acceptability area, which is $[-\infty : 2.323]$.

7.5.1. Tukey Honestly Significant Difference (HSD) Test

The Tukey HSD test is often used in conjunction with ANOVA to perform post hoc pairwise comparisons between groups to determine which group means are significantly different from each other. Table 10 depicts the fact that the mean of the pairs QGSA - DE, QGSA - GA, QGSA - PSO, QGSA - WOA, QGSA - GSA, QGSA - DRL, DE - GA, DE - PSO, DE - WOA, DE - GSA, DE - DRL, PSO - WOA, PSO - GSA, PSO - DRL, WOA - GSA, WOA - DRL, GSA - DRL are significantly different.

Table 10: Result of Tukey HSD performed on scenario-2 with 100 tasks (subsection: 7.5.1)

Pair	Difference	Standard Error of the Difference (SE)	HSD Statistics (Q)	Lower Confidence Interval	Upper Confidence Interval	Critical Mean	p-value
QGSA-WOA	0.2316	0.01322	17.5283	0.1738	0.2895	0.05785	1.27×10^{-11}
QGSA-DE	0.199	0.013	15.078	0.141	0.257	0.057	1.6×10^{-11}
QGSA-GA	0.171	0.0132	12.942	0.113	0.228	0.057	3.19×10^{-10}
QGSA-PSO	0.272	0.013	20.61	0.214	0.330	0.057	1.27×10^{-11}
QGSA-GSA	0.01486	0.013	1.124	-0.043	0.072	0.057	0.984
QGSA-DRL	0.14	0.013	10.647	0.082	0.198	0.057	5.26×10^{-8}
DE-GA	0.028	0.013	2.136	-0.029	0.086	0.057	0.736
DE-PSO	0.073	0.013	5.532	0.015	0.131	0.057	0.005
DE-WOA	0.03	0.013	2.449	-0.025	0.09	0.057	0.599
DE-GSA	0.184	0.013	13.95	0.126	0.242	0.057	4.74×10^{-11}
DE-DRL	0.058	0.013	4.43	0.0007	0.116	0.057	0.045
GA-PSO	0.101	0.013	7.668	0.043	0.159	0.057	5.21×10^{-5}
GA-WOA	0.06	0.013	4.586	0.002	0.118	0.057	0.034
GA-GSA	0.156	0.013	11.81	0.098	0.214	0.057	3.7×10^{-9}
GA-DRL	0.03	0.013	2.294	-0.027	0.088	0.057	0.669
PSO-WOA	0.04073	0.013	3.082	-0.017	0.098	0.057	0.032
WOA-GSA	0.216	0.013	16.403	0.158	0.274	0.057	1.29×10^{-11}
WOA-DRL	0.09	0.013	6.88	0.033	0.148	0.057	0.0003
PSO-GSA	0.257	0.013	19.486	0.199	0.315	0.057	1.27×10^{-11}
PSO-DRL	0.131	0.013	9.963	0.073	0.189	0.057	2.55×10^{-7}
GSA-DRL	0.125	0.013	9.523	0.068	0.183	0.057	7.07×10^{-7}

Table 11: Fitness with respect to iterations (subsection: 7.6)

Iterations:	1	50	100	150	200	250	300
QGSA	0.69294	0.69294	0.69031	0.69031	0.69006	0.68595	0.68595
GA	0.91182	0.81789	0.76933	0.74305	0.72152	0.71956	0.70367
DE	0.70622	0.71475	0.74668	0.75127	0.75281	0.74744	0.75579
WOA	0.78666	0.78417	0.78140	0.78477	0.78619	0.78348	0.78857
PSO	0.90865	0.90396	0.90489	0.90298	0.90298	0.90298	0.90298
GSA	0.89977	0.71231	0.70640	0.69305	0.69153	0.69153	0.68633

7.6. Convergence analysis

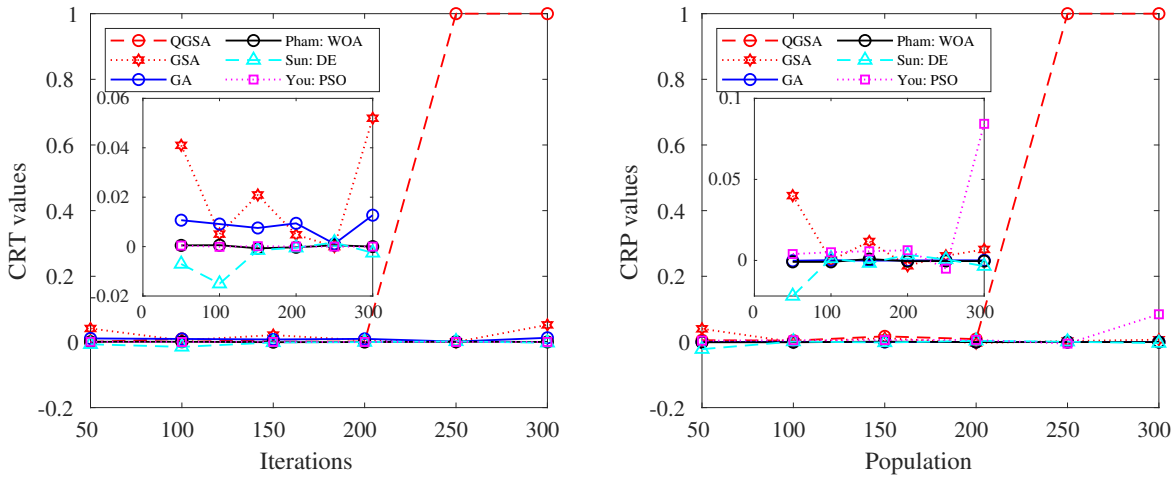
The alternative average convergence (AACR) (Ghosh and Kuila, 2023; Chen and He, 2021) rate is calculated for QGSA, GA, DE, WOA, PSO, and GSA as per the eq. (45).

$$AACR = 1 - \left| \frac{f_{t+\tau_0} - f_t}{f_t - f_{t-\tau_0}} \right|^{\frac{1}{\tau_0}}, \text{if } t \geq \tau_0 \tag{45}$$

In the context of our study, t denotes the current iteration, τ_0 denotes a predefined number of iterations (say 50 iterations), and $f_{t+\tau_0}$ denotes the fitness value in the $(t+\tau_0)$ iteration. Similarly, fitness at t and $(t-\tau_0)$ iterations are denoted by f_t and $f_{t-\tau_0}$, respectively. Using Table 11, the AACR values are calculated and put in Table 12. *1 refers to the fact that the best fitness value is not achieved but the AACR value is shown as 1. # refers to the fact when $f_t = f_{t-\tau_0}$ the denominator becomes zero while calculating AACR. Δ refers that even after reaching best fitness value, as $f_t = f_{t-\tau_0}$, the denominator becomes zero. Because of these, this research has proposed two new techniques, convergence rate concerning iteration and convergence rate concerning population where the equations are declared so that the problem raised while calculating AACR will not rise.

Table 12: AACR values (subsection: 7.6)

Iterations:	50	100	150	200	250	300
QGSA	*1	#	*1	0.05424	Δ	Δ
GA	0.01310	0.01220	0.00397	0.01582	0.04688	-0.04283
DE	-0.02673	0.03805	0.02151	-0.02519	-0.00886	0.00704
WOA	-0.00218	-0.00391	0.01711	-0.01302	*1	#
PSO	0.03184	-0.01453	*1	#	#	#
GSA	-0.07158	0.01616	-0.04437	#	*1	#



(a) (b)

Fig. 9: Convergence analysis, (a) CRT and (b) CRP.

7.6.1. Convergence rate concerning iteration

Here, the AACR equation has been modified. This is because if there comes a scenario, where $f_t = f_{t-\tau_0}$, then the equation gives an undefined result. Also, it can be noticed that if $f_t = f_{t+\tau_0}$, it gives the result as 1, even though the best fitness value is not achieved. So, the $f_{t+\tau_0}$ is replaced by the best fitness value (f_{best}) that is got while running it for a maximum number of iterations. For the minimization problem, it will be the lowest fitness value and for the maximization problem, it will be the maximum fitness value. The maximum number of iterations ran in this experiment is

350. The best result was received from QGSA as per Table 11. The convergence rate to iteration (CRT) is defined in eq. (46).

$$CRT = \begin{cases} \left(1 - \left|\frac{f_{best} - f_t}{f_{best} - f_{t-\tau_0}}\right|^{\frac{1}{\tau_0}}\right), & \text{if } (t \geq \tau_0 \text{ and } f_{best} \neq f_{t-\tau_0} \text{ or } f_t \neq f_{best}) \\ 1, & \text{if } (t \geq \tau_0 \text{ and } f_{best} = f_{t-\tau_0} \text{ or } f_t = f_{best}) \end{cases} \tag{46}$$

Here, 450 tasks were offloaded based on scenario 3. The total population size used for all the algorithms was 50. The fitness score till 300 iterations is displayed in Table 11. The fitness values for the iterations 50, 100, 150, 200, 250, and 300 are taken into account while computing the CRT. Table 13 shows that, in contrast to previous methods, QGSA has attained convergence at around 250 iterations.

Table 13: CRT values (subsection: 7.6.1)

Iterations:	50	100	150	200	250	300
QGSA	0.0	0.00937	0.00119	0.00119	1	1
GA	0.01069	0.00913	0.00754	0.00942	0.00112	0.01272
DE	-0.00705	-0.01502	-0.00145	-0.00046	0.00167	-0.00254
WOA	0.00049	0.00057	-0.00069	-0.00028	0.00054	0
PSO	0.00042	-8.502×10^{-5}	0.00017	0	0	0
GSA	0.04099	0.00506	0.02091	0.00481	0	0.05196

7.6.2. Convergence rate concerning population

This research introduces a new concept of finding the convergence rate concerning population size. The convergence rate concerning population (CRP) is determined as per eq. (47). In this research, n_0 relates to the population size of 50, N_P refers to the current population size, $f_{N_P+n_0}$ refers to the best fitness value in $(N_P + n_0)$ population size. Similarly, f_{N_P}, $f_{N_P-n_0}$ refers to best fitness value for population size N_P and $(N_P - n_0)$ respectively. The maximum population size used in this experiment is 300. The best result was received from QGSA as per Table 11.

$$CRP = \begin{cases} \left(1 - \left|\frac{f_{best} - f_{N_P}}{f_{best} - f_{N_P-n_0}}\right|^{\frac{1}{n_0}}\right), & \text{if } (N_P \geq f_{n_0} \text{ and } f_{best} \neq f_{N_P-n_0} \text{ or } f_{N_P} \neq f_{best}) \\ 1, & \text{if } (N_P \geq f_{n_0} \text{ and } f_{best} = f_{N_P-n_0} \text{ or } f_{N_P} = f_{best}) \end{cases} \tag{47}$$

Here, 450 tasks were offloaded based on scenario 3. The total iteration used for all the algorithms was 50. The fitness value up to 300 population size is displayed in Table 14. The ideal fitness values for populations of 50, 100, 150, 200, 250, and 300 are taken into account while computing the CRP. Table 15 shows that, in contrast to previous methods, QGSA has obtained convergence with a population size of 250. From Table 13 and Table 15, it is clear that the QGSA has reached convergence in 200 iterations and population. Thus, QGSA has better convergence capability than other compared algorithms.

Table 14: Best fitness with respect to population size (subsection: 7.6.2)

Population size:	1	50	100	150	200	250	300
QGSA	0.71373	0.70731	0.70329	0.69328	0.69069	0.68595	0.68595
GA	0.89577	0.89790	0.89616	0.89740	0.89752	0.89704	0.89652
DE	0.69868	0.72365	0.72141	0.72447	0.71835	0.71732	0.72336
WOA	0.77821	0.78236	0.78672	0.78255	0.78483	0.78666	0.78879
PSO	0.91085	0.87046	0.82917	0.79374	0.76452	0.78721	0.68719
GSA	0.89977	0.71373	0.71373	0.70137	0.70413	0.70169	0.69714

Table 15: CRP values (subsection: 7.6.2)

Population:	50	100	150	200	250	300
QGSA	0.00524	0.00415	0.01708	0.00864	1	1
GA	-0.00020	0.00016	-0.00011	-1.148×10^{-5}	4.58×10^{-5}	4.92×10^{-5}
DE	-0.02195	0.00122	-0.00165	0.00345	0.00064	-0.00352
WOA	-0.00088	-0.00088	0.00084	-0.00046	-0.00036	-0.00041
PSO	0.003951	0.00505	0.00566	0.00630	-0.00508	0.08430
GSA	0.03999	0.0	0.01170	-0.00329	0.00287	0.00680

8. Conclusion

The study proposes a QGSA-based method to solve the BTOP for multi-user-based, multi-UAV-aided ECS. Encoding, decoding, and linear hashing techniques are used while designing QGSA. Delay, energy, price, and resource utilization models are used to design the fitness function. Multiple scenarios are built, and the proposed QGSA is compared with various other related works. Along with that, using the proposed fitness, conventional GA is also compared with the proposed QGSA. The performance of the QGSA is found to be better. Thus, the effectiveness and broad application of the proposed work are established.

The proposed model demonstrates various applications, including disaster response, surveillance, precision agriculture, traffic management in smart cities, and more. In areas affected by disasters, UAVs equipped with edge computing capabilities can conduct real-time image and video analysis for damage assessment and surveillance. The implementation of BTO helps to offload specific tasks related to image processing and analysis, optimizing resource utilization. This UAV-enabled model is also beneficial in precision agriculture, where it captures and analyzes data concerning crop health, soil conditions, and irrigation requirements.

However, this study does not consider fault tolerance in the operation of the BTO model. The study also assumes uninterrupted UAV flights. Enhancing the energy efficiency of the BTO model could involve addressing path planning for UAVs.

References

Bandyopadhyay, B., Kuila, P., Bey, M., and Govil, M. C. (2024). Quantum-inspired differential evolution for freshness-aware caching-aided offloading in digital twin-enabled Internet of Vehicles. *IEEE Transactions on Intelligent Vehicles*, pages 1–13.

Bey, M., Kuila, P., and Naik, B. B. (2024a). Quantum-inspired differential evolution with decoding using hashing for efficient user allocation in edge computing environment. *IEEE Transactions on Emerging Topics in Computing*, pages 1–15.

Bey, M., Kuila, P., Naik, B. B., and Ghosh, S. (2024b). Quantum-inspired particle swarm optimization for efficient iot service placement in edge computing systems. *Expert Systems with Applications*, 236:121270.

Bozkaya, E. (2023). Digital twin-assisted and mobility-aware service migration in Mobile Edge Computing. *Computer Networks*, 231:109798.

Bozkaya, E., Erel-Özçevik, M., Bilen, T., and Özçevik, Y. (2023). Proof of Evaluation-based energy and delay aware computation offloading for Digital Twin Edge Network. *Ad Hoc Networks*, 149:103254.

Chen, Y. and He, J. (2021). Average convergence rate of evolutionary algorithms in continuous optimization. *Information Sciences*, 562:200–219.

Dhingan, D., Ghosh, S., Naik, B. B., and Kuila, P. (2023). Energy and Delay Efficient Partial Offloading for UAV-assisted MEC Systems using Differential Evolution Algorithm. In *2023 Third International Conference on Secure Cyber Computing and Communication (ICSCCC)*, pages 415–420. IEEE.

Fan, Y., Wang, L., Wu, W., and Du, D. (2021). Cloud/Edge Computing Resource Allocation and Pricing for Mobile Blockchain: An Iterative Greedy and Search Approach. *IEEE Transactions on Computational Social Systems*, 8(2):451–463.

Farimani, M. K., Karimian-Aliabadi, S., Entezari-Maleki, R., Egger, B., and Sousa, L. (2024). Deadline-aware task offloading in vehicular networks using deep reinforcement learning. *Expert Systems with Applications*, 249:123622.

Ghosh, S. and Kuila, P. (2023). Efficient offloading in disaster-affected areas using unmanned aerial vehicle-assisted mobile edge computing: A gravitational search algorithm-based approach. *International Journal of Disaster Risk Reduction*, 97:104067.

Gupta, A. K., Ghosh, S., and Bhatnagar, M. R. (2022). Pricing Based Scheme for UAV-Enabled Wireless Energy Transfer. *IEEE Transactions on Vehicular Technology*, 71(12):13198–13209.

Han, D., Chen, W., and Fang, Y. (2018). A Dynamic Pricing Strategy for Vehicle Assisted Mobile Edge Computing Systems. *IEEE Wireless Communications Letters*, 8(2):420–423.

Han, K.-H. and Kim, J.-H. (2002). Quantum-inspired evolutionary algorithm for a class of combinatorial optimization. *IEEE Transactions on Evolutionary Computation*, 6(6):580–593.

He, Y., Zhai, D., Huang, F., Wang, D., Tang, X., and Zhang, R. (2021). Joint Task Offloading, Resource Allocation, and Security Assurance for Mobile Edge Computing-Enabled UAV-Assisted VANETs. *Remote Sensing*, 13(8):1547.

Hortelano, D., de Miguel, I., Barroso, R. J. D., Aguado, J. C., Merayo, N., Ruiz, L., Asensio, A., Masip-Bruin, X., Fernández, P., Lorenzo, R. M., et al. (2023). A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems. *Journal of Network and Computer Applications*, 216:103669.

Huang, L., Feng, X., Zhang, C., Qian, L., and Wu, Y. (2019). Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing. *Digital Communications and Networks*, 5(1):10–17.

Huda, S. A. and Moh, S. (2022). Survey on computation offloading in UAV-Enabled mobile edge computing. *Journal of Network and Computer Applications*, 201:103341.

Huynh, L. N., Pham, Q.-V., Nguyen, T. D., Hossain, M. D., Park, J. H., and Huh, E.-N. (2020). A Study on Computation Offloading in MEC Systems using Whale Optimization Algorithm. In *2020 14th International Conference on Ubiquitous Information Management and Communication (IMCOM)*, pages 1–4. IEEE.

Islam, A., Debnath, A., Ghose, M., and Chakraborty, S. (2021). A Survey on Task Offloading in Multi-access Edge Computing. *Journal of Systems Architecture*, 118:102225.

Kuila, P. and Jana, P. K. (2014). Energy efficient clustering and routing algorithms for wireless sensor networks: Particle swarm optimization approach. *Engineering Applications of Artificial Intelligence*, 33:127–140.

Li, S., Hu, X., and Du, Y. (2021). Deep Reinforcement Learning for Computation Offloading and Resource Allocation in Unmanned-Aerial-Vehicle Assisted Edge Computing. *Sensors*, 21(19):6499.

Li, X., Zhou, L., Sun, Y., and Ulziinyam, B. (2020). Multi-task offloading scheme for UAV-enabled fog computing networks. *EURASIP Journal on Wireless Communications and Networking*, 2020(1):1–16.

Li, Z. and Zhu, Q. (2020). Genetic Algorithm-Based Optimization of Offloading and Resource Allocation in Mobile-Edge Computing. *Information*, 11(2):83.

Liao, L., Lai, Y., Yang, F., and Zeng, W. (2023). Online computation offloading with double reinforcement learning algorithm in mobile edge computing. *Journal of Parallel and Distributed Computing*, 171:28–39.

Liao, Y., Qiao, X., Yu, Q., and Liu, Q. (2021). Intelligent dynamic service pricing strategy for multi-user vehicle-aided MEC networks. *Future Generation Computer Systems*, 114:15–22.

Liu, W., Li, B., Xie, W., Dai, Y., and Fei, Z. (2023). Energy Efficient Computation Offloading in Aerial Edge Networks With Multi-Agent Cooperation. *IEEE Transactions on Wireless Communications*.

Ma, M. and Wang, Z. (2023). Distributed Offloading for Multi-UAV Swarms in MEC-Assisted 5G Heterogeneous Networks. *Drones*, 7(4):226.

Nandi, P. K., Reaj, M. R. I., Sarker, S., Razzaque, M. A., Mamun-or Rashid, M., and Roy, P. (2024). Task offloading to edge cloud balancing utility and cost for energy harvesting Internet of Things. *Journal of Network and Computer Applications*, 221:103766.

Qiao, B., Liu, C., Liu, J., Hu, Y., Li, K., and Li, K. (2022). Task migration computation offloading with low delay for mobile edge computing in vehicular networks. *Concurrency and Computation: Practice and Experience*, 34(1):e6494.

Qiu, J., Grace, D., Ding, G., Yao, J., and Wu, Q. (2019). Blockchain-Based Secure Spectrum Trading for Unmanned-Aerial-Vehicle-Assisted Cellular Networks: An Operator's Perspective. *IEEE Internet of Things Journal*, 7(1):451–466.

Ram, P. K. and Kuila, P. (2023). GAAE: a novel genetic algorithm based on autoencoder with ensemble classifiers for imbalanced healthcare data. *The Journal of Supercomputing*, 79(1):541–572.

Saad, H. M., Chakrabortty, R. K., Elsayed, S., and Ryan, M. J. (2021). Quantum-Inspired Genetic Algorithm for Resource-Constrained Project-Scheduling. *IEEE Access*, 9:38488–38502.

Song, F., Xing, H., Luo, S., Zhan, D., Dai, P., and Qu, R. (2020). A multiobjective computation offloading algorithm for mobile-edge computing. *IEEE Internet of Things Journal*, 7(9):8780–8799.

Song, S., Ma, S., Yang, L., Zhao, J., Yang, F., and Zhai, L. (2022). Delay-sensitive tasks offloading in multi-access edge computing. *Expert Systems with Applications*, 198:116730.

Song, S., Ma, S., Zhu, X., Li, Y., Yang, F., and Zhai, L. (2023). Joint bandwidth allocation and task offloading in multi-access edge computing. *Expert Systems with Applications*, 217:119563.

Sun, Y., Song, C., Yu, S., Liu, Y., Pan, H., and Zeng, P. (2021). Energy-Efficient Task Offloading Based on Differential Evolution in Edge Computing System With Energy Harvesting. *IEEE Access*, 9:16383–16391.

Tang, M. and Wong, V. W. (2020). Deep Reinforcement Learning for Task Offloading in Mobile Edge Computing Systems. *IEEE Transactions on Mobile Computing*, 21(6):1985–1997.

Thakur, A. S., Biswas, T., and Kuila, P. (2018). Gravitational search algorithm based task scheduling for multi-processor systems. In *2018 Fourth International Conference on Research in Computational Intelligence and Communication Networks (ICRCICN)*, pages 253–257. IEEE.

Thakur, A. S., Biswas, T., and Kuila, P. (2021). Binary quantum-inspired gravitational search algorithm-based multi-criteria scheduling for multi-processor computing systems. *The Journal of Supercomputing*, 77:796–817.

Vardakas, J. S., Zorba, N., and Verikoukis, C. V. (2014). A Survey on Demand Response Programs in Smart Grids: Pricing Methods and Optimization Algorithms. *IEEE Communications Surveys & Tutorials*, 17(1):152–178.

Wang, Y., Fang, W., Ding, Y., and Xiong, N. (2021). Computation offloading optimization for UAV-assisted mobile edge computing: a deep deterministic policy gradient approach. *Wireless Networks*, 27(4):2991–3006.

Wu, G., Miao, Y., Zhang, Y., and Barnawi, A. (2020). Energy efficient for UAV-enabled mobile edge computing networks: Intelligent task prediction and offloading. *Computer Communications*, 150:556–562.

Xu, D. and Xu, D. (2023). Cooperative task offloading and resource allocation for UAV-enabled mobile edge computing systems. *Computer Networks*, page 109574.

You, Q. and Tang, B. (2021). Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. *Journal of Cloud Computing*, 10(1):1–11.

Yu, Z., Gong, Y., Gong, S., and Guo, Y. (2020). Joint task offloading and resource allocation in UAV-enabled mobile edge computing. *IEEE Internet of Things Journal*, 7(4):3147–3159.

Yuan, H., Bi, J., Wang, Z., Yang, J., and Zhang, J. (2024). Partial and cost-minimized computation offloading in hybrid edge and cloud systems. *Expert Systems with Applications*, 250:123896.

Zeng, C., Wang, X., Zeng, R., Li, Y., Shi, J., and Huang, M. (2024). Joint optimization of multi-dimensional resource allocation and task offloading for QoE enhancement in Cloud-Edge-End collaboration. *Future Generation Computer Systems*, 155:121–131.

Zeng, Y., Xu, J., and Zhang, R. (2019). Energy minimization for wireless communication with rotary-wing UAV. *IEEE Transactions on Wireless Communications*, 18(4):2329–2345.

Zhang, G. (2011). Quantum-inspired evolutionary algorithms: a survey and empirical study. *Journal of Heuristics*, 17(3):303–351.

Zhang, J., Zhou, L., Tang, Q., Ngai, E. C.-H., Hu, X., Zhao, H., and Wei, J. (2018). Stochastic computation offloading and trajectory scheduling for UAV-assisted mobile edge computing. *IEEE Internet of Things Journal*, 6(2):3688–3699.

Zhang, T., Xu, Y., Loo, J., Yang, D., and Xiao, L. (2019). Joint computation and communication design for UAV-assisted mobile edge computing in IoT. *IEEE Transactions on Industrial Informatics*, 16(8):5505–5516.

Zhang, X., Wu, W., Liu, S., and Wang, J. (2023). An efficient computation offloading and resource allocation algorithm in RIS empowered MEC. *Computer Communications*, 197:113–123.

Zhang, Y. and Yan, L. (2023). Research on the optimization of energy consumption for multi-priority tasks in mobile computing offloading. *Multimedia Tools and Applications*, 82(29):45453–45469.

Zhao, Z., Zhao, R., Xia, J., Lei, X., Li, D., Yuen, C., and Fan, L. (2019). A novel framework of three-hierarchical offloading optimization for MEC in industrial IoT networks. *IEEE Transactions on Industrial Informatics*, 16(8):5424–5434.

Zhu, A. and Wen, Y. (2021). Computing Offloading Strategy Using Improved Genetic Algorithm in Mobile Edge Computing System. *Journal of Grid Computing*, 19(3):1–12.

Zolghadri, M., Asghari, P., Dashti, S., and Hedayati, A. (2024). Resource allocation in Fog-Cloud Environments: State of the art. *Journal of Network and Computer Applications*, page 103891.